Advanced Software Engineering Lab



CENTRAL UNIVERSITY OF KARNATAKA

MASTER OF COMPUTER APPLICATIONS

Submitted by
Aditya Shekhar (25PCOMP003)
PG 2ND YEAR MCA (2025-27)

Under the guidance of
Dr. Rohit Kumar B
Assistant Professor
School of Computer Science

SCHOOL OF COMPUTER SCIENCE
CENTRAL UNIVERSITY OF KARNATAKA, KADAGANCHI,
ALAND ROAD, KALABURAGI DISTRICT, KARNATAKA – 585367, INDIA

Certificate

This is to certify that the laboratory record titled **“Advanced Software Engineering Lab – Campus Connect”** is the bonafide work of Team FADED (Subham Kumar Bhoi, Aditya Shekhar, Ashish Kumar and Sovit Kumar Mahakur), carried out in the School of Computer Science, Central University of Karnataka, Kalaburagi, under the guidance of Dr. Rohit Kumar B., and that it contains the fifteen experiments prescribed for the course.

Course Instructor	Head of Department
Dr. Rohit Kumar B.	

Before you submit: items to personalise

- Cover page: your name, roll number, programme, semester and year.
- Role allocation (Experiments 2, 10, 14): adjust to who actually did what.
- Test execution tables (Experiments 11 and 12): fill the Actual / Status columns after running the tests.
- Defect log and FTR record (Experiment 14) are illustrative; replace with your real review findings.
- Git history sample and repository URL (Experiment 13): use your real repository.
- Screenshots of the running Campus_Connect.html: paste next to the diagrams where the lab record format asks for them (diagrams here are drawn, not captured).
- Experiment 11 uses a representative filter function; match it to the function names in your own HTML file.

Index

Experiment 1 – Study of Software Process Models 4

Experiment 2 – Software Project Planning (SPMP, WBS, Gantt Chart) 8

Experiment 3 – Software Cost Estimation (LOC, FP, COCOMO) 12

Experiment 4 – Risk Analysis 16

Experiment 5 – Software Requirement Specification (SRS) 20

Experiment 6 – UML Modeling 24

Experiment 7 – Design Modeling 30

Experiment 8 – Architecture Design 35

Experiment 9 – Interface Design 39

Experiment 10 – Test Planning 43

Experiment 11 – White Box Testing 48

Experiment 12 – Black Box Testing 53

Experiment 13 – Configuration Management (Git and GitHub) 57

Experiment 14 – Software Quality Assurance 61

Experiment 15 – Mini Project – Documentation and Presentation 65

Course outcome coverage

Outcome	Statement	Experiments
CO1	Apply software engineering process models in software development	1, 15
CO2	Perform project planning, estimation and risk analysis	2, 3, 4
CO3	Develop analysis and design models using UML and CASE tools	5, 6, 7, 8, 9
CO4	Design and execute software testing strategies and test cases	10, 11, 12
CO5	Apply software quality assurance and configuration management practices	13, 14

Lab record format

Each experiment follows the prescribed format: 1 Aim, 2 Objectives, 3 Theory, 4 Software / Tools Required, 5 Procedure, 6 Application to Campus Connect (diagrams, tables, calculations), 7 Observations, 8 Result, 9 Viva Questions and Answers.

EXPERIMENT 1

Study of Software Process Models

Lab week	Week 1 – 2
Course outcome	CO1
Tools used	MS Office, Draw.io, Lucidchart
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To study the software development life cycle (SDLC) and the Waterfall, Prototype, RAD, Spiral and Agile process models, and to select and justify the most suitable model for developing **Campus Connect**.

2. Objectives

- Understand the generic SDLC activities and the purpose of a process model.
- Compare Waterfall, Prototype, RAD, Spiral and Agile / Incremental models on common criteria.
- Select a real-world project (Campus Connect), identify the suitable model and justify the choice with a weighted decision matrix.
- Prepare a comparison report as the deliverable of this experiment.

3. Theory

A **software process model** describes the order of development activities, the work products each activity produces and the review points between them. Pressman describes a generic framework of five activities that every model arranges differently: *communication, planning, modelling, construction* and *deployment*. Sommerville groups the models into plan-driven (e.g. Waterfall) and incremental / agile approaches.

Model	Core idea	Strengths	Weaknesses	Use when
Waterfall	Phases run strictly one after another: requirements, design, code, test, deploy, maintain.	Simple; clear milestones; good documentation.	No working software until late; changes are costly.	Requirements are stable and well understood.
Prototype	Build a quick mock-up, let users evaluate it, refine until requirements are agreed (throw-away or evolutionary).	Early feedback; fewer requirement errors.	Users may mistake a prototype for the product; quick-and-dirty design may stay.	Requirements are unclear; UI-heavy systems.
RAD	Very short cycle (60–90 days); component-based construction by parallel teams.	Fast delivery; reuse of components.	Needs a modular system, skilled people and several teams.	Business systems that can be split into modules.
Spiral	Risk-driven loops: set objectives, analyse risk, develop and test, plan the next loop.	Explicit risk management; flexible.	Needs risk expertise; heavy management overhead.	Large, high-risk projects.
Agile / Incremental	Short iterations deliver working increments; continuous customer collaboration (Scrum, XP).	Handles change well; frequent working releases.	Needs steady user involvement; light documentation.	Changing requirements; small teams.

4. Software / Tools Required

- Microsoft Word / Excel for the report and the decision matrix.
- Draw.io or Lucidchart for process diagrams.

5. Procedure

1. Study the five process models and note their phases, strengths and limitations.
2. Select the real-world project: Campus Connect (campus marketplace plus career portal).
3. List the project characteristics that influence the choice (requirement stability, schedule, team, risk, documentation).

4. Describe how each model would be applied to the project.
5. Score every model against weighted criteria (scale 1 = poor, 5 = excellent) and calculate the weighted score.
6. Select the model with the highest score, justify it and draw its life-cycle diagram.

6. Application to Campus Connect

6.1 Project characteristics

Characteristic	Campus Connect
Product	Responsive web application with two modules: Campus Marketplace and Career Connect, plus accounts, dashboard, notifications and light / dark theme.
Team	Four students (Team FADED) guided by Dr. Rohit Kumar B.; limited industrial experience.
Time	One semester (15 lab weeks).
Requirements	Core idea is clear, but details such as application statuses, dashboard contents and filters were refined while building.
Deliverable	Working HTML prototype that can be demonstrated early and extended later.

6.2 How each model would run on this project

Model	Application to Campus Connect	Fit
Waterfall	Write the full SRS for both modules, design everything, then code the whole site, then test. The marketplace and career portal would be seen working only at the end.	Poor – details changed during build.
Prototype	Build a clickable HTML prototype of marketplace and careers, review it with the guide and classmates, refine screens and rules, repeat.	Good for UI, weak on planning.
RAD	Split into accounts, marketplace, career and dashboard components, build in parallel in a 60–90 day time-box.	Moderate – team too small; one shared file raises merge conflicts.
Spiral	Loop 1: concept and data-persistence risk; Loop 2: prototype; Loop 3: build and test; Loop 4: release and review.	Good risk control, heavy overhead for a mini project.
Incremental / Agile	Deliver four increments (shell, marketplace, career, dashboard); each ends with a demo and feedback that shape the next.	Best fit.

6.3 Weighted decision matrix (score 1–5)

Criterion	Weight %	Waterfall	Prototype	RAD	Spiral	Incr. / Agile
Requirements may change during development	15	1	4	3	4	5
Early working version for feedback	20	1	5	4	3	5
Explicit risk handling	10	2	3	2	5	3
Small team of four students, limited experience	15	4	4	2	2	4
Short fixed schedule (one semester)	15	3	4	4	2	5
Documentation needed for academic evaluation	10	5	2	3	4	3
Ability to absorb change late	15	1	4	3	4	5
Weighted score (out of 5)	**100**	**2.25**	**3.90**	**3.10**	**3.30**	**4.45**

Table 1.1 Weighted score = Σ (weight × score) / 100. Example for Waterfall: (15×1 + 20×1 + 10×2 + 15×4 + 15×3 + 10×5 + 15×1) / 100 = 2.25.

6.4 Selected model: Incremental development with Agile practices and prototyping

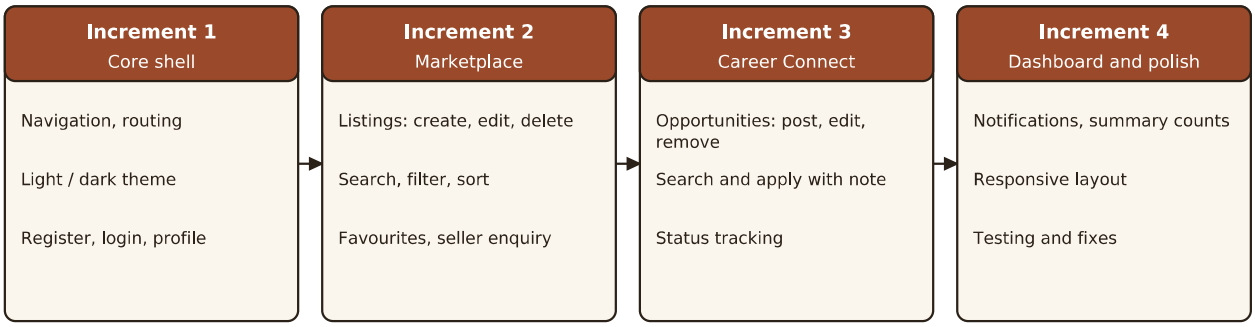


Fig. 1.1 Incremental life cycle planned for Campus Connect

Justification.

- The final deliverable is itself a working prototype, so frequent demonstrations are natural and give early feedback.
- Requirement details (statuses, filters, dashboard counts) evolved during development; short increments absorb such change cheaply.
- A team of four with a one-semester deadline needs small, testable pieces rather than a single large hand-over.
- Risk is handled informally through the risk register (Experiment 4) and a review at the end of every increment.
- Documentation required by the lab (SRS, UML models, test plan) is produced incrementally alongside the code.

7. Observations

- Waterfall scored lowest (2.25) because it delays working software and resists change.
- Prototype (3.90) and Spiral (3.30) scored well on feedback and risk, but Prototype alone lacks planning and Spiral adds overhead.
- RAD (3.10) is limited by team size and the single-file architecture.
- Incremental / Agile obtained the highest score (4.45 out of 5).

8. Result

The Incremental model with Agile practices and throw-away prototypes was identified as the most suitable process model for Campus Connect, and the choice was justified using a weighted comparison of five models.

9. Viva Questions and Answers

Q1. What is an SDLC and why is it needed?

Ans. The SDLC is the structured sequence of activities (planning, analysis, design, coding, testing, deployment, maintenance) used to build software. It brings predictability, quality and control to development.

Q2. How does the Waterfall model differ from the Spiral model?

Ans. Waterfall is linear and completes each phase before the next, with no formal risk analysis. Spiral repeats four steps in loops and is driven by risk analysis at every loop.

Q3. What is the difference between throw-away and evolutionary prototyping?

Ans. A throw-away prototype is built only to clarify requirements and is discarded; an evolutionary prototype is refined step by step until it becomes the final system.

Q4. Why is RAD not the best choice for Campus Connect?

Ans. RAD needs a well-modularised system, multiple skilled teams and a strict 60–90 day time-box. Our team has four members and one shared HTML file, so parallel work causes integration problems.

Q5. How are Incremental and Agile models related?

Ans. Incremental delivers the product in parts. Agile adds practices such as short iterations, daily communication, customer collaboration and welcoming change. Agile projects are normally incremental.

Q6. Which model did you select and why?

Ans. Incremental with Agile practices, because it scored highest (4.45 / 5): it gives early working software, absorbs changing requirements and suits a small team with a fixed semester.

Q7. What are the five generic framework activities given by Pressman?

Ans. Communication, planning, modelling, construction and deployment.

EXPERIMENT 2

Software Project Planning (SPMP, WBS, Gantt Chart)

Lab week	Week 3
Course outcome	CO2
Tools used	MS Project / ProjectLibre, Excel, Draw.io
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To prepare a Software Project Management Plan (SPMP) for Campus Connect covering the problem statement, objectives, scope, feasibility analysis, work breakdown structure, Gantt chart and milestones.

2. Objectives

- Define the problem, objectives and scope of the project.
- Perform technical, economic, operational, schedule and legal feasibility analysis.
- Apply the W5HH principle to organise the plan.
- Create a WBS, Gantt chart and milestone list.

3. Theory

Project planning establishes *what* will be built, *who* will build it, *when* and *at what cost*. The **SPMP** (IEEE Std 1058) records the project organisation, managerial process, technical process and work packages. Boehm's **W5HH principle** asks seven questions: *Why* is the system being built? *What* will be done? *When*? *Who* is responsible? *Where* are they located? *How* will the work be done? *How much* of each resource is needed?

A **Work Breakdown Structure (WBS)** decomposes the project into manageable work packages. A **Gantt chart** shows task durations on a calendar, and a **milestone** is a checkpoint marking completion of a significant deliverable. A **feasibility study** checks whether the project is worth doing from technical, economic, operational, schedule and legal viewpoints.

4. Software / Tools Required

- MS Project or ProjectLibre for WBS and Gantt chart (Excel can be used as an alternative).
- MS Word for the SPMP document; Draw.io for diagrams.

5. Procedure

1. Write the problem statement and objectives from the project vision.
2. Define scope: what is included and what is excluded.
3. Analyse the five types of feasibility.
4. Answer the W5HH questions and allocate team roles.
5. Decompose the project into a WBS (levels 1 and 2).
6. Estimate durations, sequence tasks and draw the Gantt chart.
7. Mark milestones and deliverables.

6. Application to Campus Connect

6.1 Problem statement

Students find it hard to locate affordable second-hand items inside their campus community and to discover internships, mentorships and project opportunities in one place. Existing options are scattered across messaging groups and notice boards, with no profiles, no manageable listings and no application tracking. Campus Connect provides one responsive web application that brings a campus marketplace and a career portal together.

6.2 Objectives

- Provide a campus marketplace with search by keyword, category and condition, price filter, sorting, personal listings, favourites and seller enquiries.
- Provide a career portal where recruiters post, edit and remove roles and students apply and track status.
- Offer shared accounts, profiles, dashboards, notifications and a light / dark theme.
- Deliver a responsive, accessible HTML prototype within one semester.

6.3 Scope

In scope	Out of scope (current prototype)
Registration, login, profile management; marketplace (browse, search, filter, sort, create / edit / delete listings, save items, enquiries); Career Connect (post / edit / remove opportunities, search, apply, review applicants, update status); dashboard; notifications; light / dark theme; sample demonstration data.	Server-side database and shared data between users; real university verification; file / image uploads; payments; real-time chat. These are listed as next-phase work.

6.4 Feasibility analysis

Type	Assessment	Result
Technical	Uses HTML, CSS, JavaScript and browser storage, all known to the team; no special hardware or server needed.	Feasible
Economic	Free tools (VS Code, Git, GitHub, Draw.io, Figma free plan); no licence or hosting cost; cost is student time only (see Experiment 3).	Feasible
Operational	Meets a real campus need; users already use web and mobile browsers; simple navigation keeps training effort low.	Feasible
Schedule	Core modules are small; four increments fit into 15 weeks with buffer for testing.	Feasible with tight control
Legal / ethical	Prototype holds only demonstration data; real deployment would need data-protection and moderation policy.	Feasible for prototype

6.5 Team organisation

Member	Role in project
Subham Kumar Bhoi (Lead)	Project manager, integration, repository owner
Aditya Shekhar	Marketplace module, testing
Ashish Kumar	Career Connect module, dashboard
Sovit Kumar Mahakur	UI / UX, documentation, quality review

Role allocation above is a suggested split for the record. Edit it to match the actual contribution of each member.

6.6 W5HH plan

Question	Answer for Campus Connect
Why is the system built?	To bring campus resale and career discovery into one trusted, easy-to-use place.
What will be done?	Marketplace and Career Connect modules with accounts, dashboards, notifications and theme.
When?	15 lab weeks; features complete by week 13, quality review and demonstration in week 14–15.
Who is responsible?	Team FADED (four students); guide: Dr. Rohit Kumar B.
Where?	School of Computer Science, Central University of Karnataka, Kalaburagi; code hosted on GitHub.
How?	Incremental development, UML modelling, HTML / CSS / JavaScript, Git branching, peer review and testing.
How much?	About 12.4 person-months of effort (Experiment 3), tools at no cost.

6.7 Work Breakdown Structure

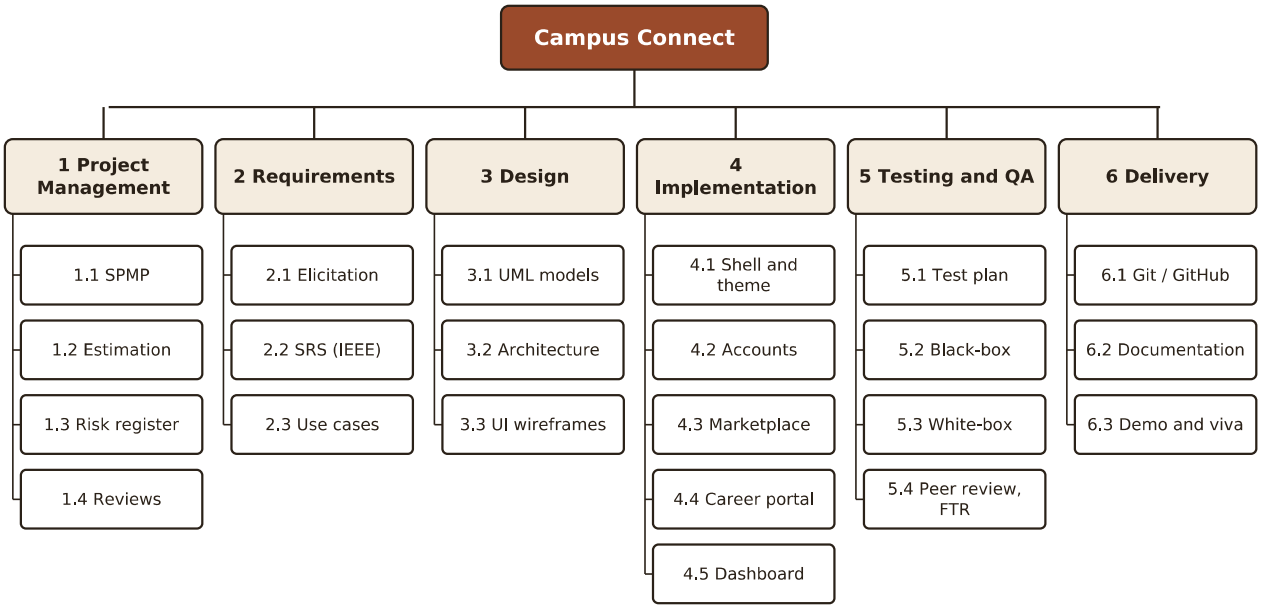


Fig. 2.1 WBS of Campus Connect (levels 1 and 2)

6.8 Gantt chart

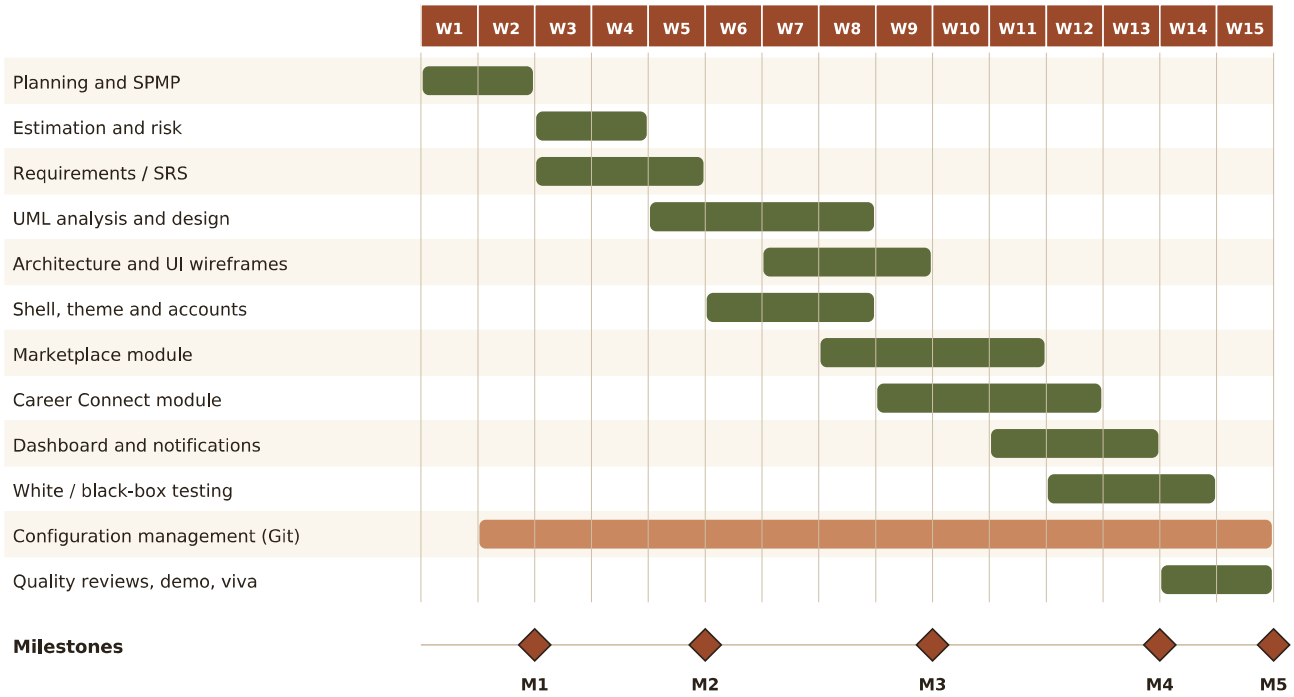


Fig. 2.2 Project schedule (15 weeks). Milestones: M1 SPMP approved (W2), M2 SRS baseline (W5), M3 design complete (W9), M4 features complete (W13), M5 final demo and viva (W15).

6.9 Milestones and deliverables

Milestone	Week	Deliverable	Completion criterion
M1 Plan approved	2	SPMP, risk register	Guide signs off scope and schedule
M2 Requirements baseline	5	IEEE SRS	All functional requirements numbered and reviewed
M3 Design complete	9	UML models, architecture, wireframes	Models consistent with SRS
M4 Feature complete	13	Working prototype, test cases	All modules run; test plan executed
M5 Final demo	15	Repository, report, presentation	Demonstration and viva completed

7. Observations

- The Gantt chart overlaps design and construction because increments are delivered iteratively.
- Configuration management runs through the whole schedule instead of being a separate phase.
- Feasibility analysis shows the project is achievable with free tools; the main constraint is time.

8. Result

A complete SPMP for Campus Connect was prepared with problem statement, objectives, scope, feasibility analysis, W5HH plan, WBS, Gantt chart and five milestones.

9. Viva Questions and Answers

Q1. What is an SPMP?

Ans. The Software Project Management Plan is the document that describes how the project will be organised, scheduled, staffed, monitored and controlled (IEEE 1058).

Q2. Explain the W5HH principle.

Ans. Boehm's set of seven questions – Why, What, When, Who, Where, How, How much – that help define project objectives, schedule, responsibilities, methods and resources.

Q3. What is a WBS and what is a work package?

Ans. A WBS is a hierarchical decomposition of the project into deliverable-oriented tasks. The lowest-level items are work packages that can be estimated and assigned.

Q4. How is a Gantt chart different from a PERT / CPM network?

Ans. A Gantt chart is a bar chart of tasks against time. PERT / CPM shows task dependencies as a network and identifies the critical path.

Q5. What is a milestone?

Ans. A checkpoint that marks completion of a major deliverable, used to verify progress against the plan.

Q6. Which feasibility factors did you study?

Ans. Technical, economic, operational, schedule and legal feasibility; all were found acceptable for the prototype.

Q7. What is excluded from the scope of Campus Connect?

Ans. Server-side database, real university verification, uploads, payments and real-time chat.

EXPERIMENT 3

Software Cost Estimation (LOC, FP, COCOMO)

Lab week	Week 4 – 5
Course outcome	CO2
Tools used	Microsoft Excel
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To estimate the size, effort, cost and development time of Campus Connect using LOC estimation, Function Point analysis and the Basic COCOMO model.

2. Objectives

- Estimate size using bottom-up LOC (three-point) estimation.
- Compute unadjusted and adjusted function points.
- Apply Basic COCOMO to obtain effort, development time and average staffing.
- Compare the estimates and derive cost and productivity figures.

3. Theory

LOC measures size as thousands of delivered source lines (KLOC). The **three-point** technique gives an expected value $E = (O + 4M + P) / 6$ from optimistic, most likely and pessimistic estimates. **Function Points (FP)**, introduced by Albrecht, measure functionality delivered to the user independent of language: *UFP* is the weighted count of external inputs, outputs, inquiries, internal logical files and external interface files; the *Value Adjustment Factor* is VAF = 0.65 + 0.01 × ΣF_i over 14 general system characteristics rated 0–5; and FP = UFP × VAF.

Basic COCOMO (Boehm, 1981) estimates Effort E = a × $(KLOC)^b$ person-months and Development time D = c × E^d months, with coefficients that depend on the project mode. Average staff = E / D.

Mode	a	b	c	d	Typical project
Organic	2.4	1.05	2.5	0.38	Small team, familiar environment, simple requirements
Semi-detached	3.0	1.12	2.5	0.35	Mixed experience, medium complexity
Embedded	3.6	1.20	2.5	0.32	Tight hardware and operational constraints

4. Software / Tools Required

- Microsoft Excel (calculation sheets).
- Calculator for verification.

5. Procedure

1. List the project modules and estimate O, M and P lines of code for each; compute expected LOC.
2. Identify and classify EI, EO, EQ, ILF and EIF; multiply by complexity weights to get UFP.
3. Rate the 14 general system characteristics, compute VAF and FP.
4. Convert FP to KLOC with a language factor (cross-check).
5. Apply Basic COCOMO (organic mode) to get effort and development time; calculate staff and cost.
6. Compare results and record observations.

6. Application to Campus Connect

6.1 LOC estimation (bottom-up, three-point)

Module	O	M	P	Expected = (O + 4M + P) / 6
Shared shell: navigation, routing, CSS, light / dark theme	600	800	1100	816.7
Accounts, authentication and profile	400	550	800	566.7
Marketplace module	900	1200	1600	1216.7
Career Connect module	900	1200	1700	1233.3
Dashboard and notifications	350	500	750	516.7
Storage helper, seed data, utilities	300	400	600	416.7
Total	**3450**	**4650**	**6550**	**4766.7 (4.77 KLOC)**

Table 3.1 LOC estimate. Lines include HTML markup, CSS and JavaScript of the single-file prototype.

6.2 Function Point analysis

Step 1 – Count of functions.

Component	Examples in Campus Connect	Simple	Average	Complex
EI (12)	Simple: login, delete listing, save / unsave item, send enquiry. Average: register, update profile, create listing, edit listing, edit / remove opportunity, apply with note. Complex: post opportunity, update application status.	4	6	2
EO (6)	Simple: listing detail, opportunity detail. Average: listing cards, opportunity cards, notification list. Complex: dashboard summary and applicant report.	2	3	1
EQ (6)	Simple: keyword search of items, search of opportunities, view saved items. Average: filter + sort items, view my applications, view my listings.	3	3	0
ILF (5)	Simple: notifications, saved items. Average: users, listings with enquiries, opportunities with applications.	2	3	0
EIF (2)	Simple: seed listings dataset, seed opportunities dataset (demonstration data).	2	0	0

Step 2 – Unadjusted function points.

Component	Simple × w	Average × w	Complex × w	Total
External Inputs (EI)	4 × 3 = 12	6 × 4 = 24	2 × 6 = 12	48
External Outputs (EO)	2 × 4 = 8	3 × 5 = 15	1 × 7 = 7	30
External Inquiries (EQ)	3 × 3 = 9	3 × 4 = 12	0 × 6 = 0	21
Internal Logical Files (ILF)	2 × 7 = 14	3 × 10 = 30	0 × 15 = 0	44
External Interface Files (EIF)	2 × 5 = 10	0 × 7 = 0	0 × 10 = 0	10
Unadjusted FP (UFP)				**153**

Step 3 – General system characteristics (rating 0–5).

#	Characteristic	Rating	#	Characteristic	Rating
1	Data communications	2	8	Online update	4
2	Distributed data processing	0	9	Complex processing	2
3	Performance	3	10	Reusability	2
4	Heavily used configuration	2	11	Installation ease	3
5	Transaction rate	3	12	Operational ease	4
6	Online data entry	5	13	Multiple sites	1
7	End-user efficiency	4	14	Facilitate change	3

$\Sigma F_i = 38$, so VAF = 0.65 + 0.01 × 38 = **1.03**.

FP = UFP × VAF = 153 × 1.03 = 157.59 ≈ 157.6 function points.

Using an average of about 47 LOC per FP for JavaScript, the FP-based size is 157.59 × 47 ≈ 7407 LOC (7.41 KLOC). This is used as a cross-check.

6.3 Basic COCOMO (organic mode)

Campus Connect is a small, in-house, well-understood project built by a small team, so the **organic** mode is chosen. Size K = 4.77 KLOC (bottom-up estimate).

```
Effort   E = 2.4 x (4.77)^1.05 = 12.37 person-months
Time     D = 2.5 x (12.37)^0.38 = 6.50 months
Staff    S = E / D = 12.37 / 6.50 = 1.90 persons
```

6.4 Cost, productivity and phase-wise effort

Assuming a notional labour rate of ₹25,000 per person-month (assumption for a student project), Cost = 12.37 × ₹25,000 = **₹3,09,227**.

Measure	Calculation	Value
Productivity (LOC / PM)	4767 / 12.37	385 LOC per person-month
Productivity (FP / PM)	157.6 / 12.37	12.7 FP per person-month
Cost per LOC	₹3,09,227 / 4767	₹65 per LOC
Cost per FP	₹3,09,227 / 157.6	₹1,962 per FP

Phase (assumed distribution)	% of effort	Person-months
Planning and requirements	12%	1.48
Design	20%	2.47
Coding	33%	4.08
Testing	25%	3.09
Documentation and deployment	10%	1.24
Total	**100%**	**12.37**

6.5 Comparison of results and sensitivity

Mode	Effort (PM), 4.77 KLOC	Time (months)	Staff	Effort (PM), 7.41 KLOC (FP-based)
Organic	12.37	6.50	1.90	19.65
Semi-detached	17.25	6.77	2.55	28.26
Embedded	23.45	6.86	3.42	39.80

7. Observations

- The bottom-up LOC estimate (4.77 KLOC) is lower than the FP-based size (7.41 KLOC). FP conversion factors are industry averages and include more markup and framework code than our hand-written prototype.
- COCOMO predicts 12.4 person-months and 6.5 months for a professional team working full time. Our four-member student team works part time, so actual calendar time is spread over the 15-week semester with fewer hours per person.
- Effort rises sharply in semi-detached and embedded modes, showing how sensitive COCOMO is to the chosen mode and size.

8. Result

Estimate	Value
Size (bottom-up)	4767 LOC = 4.77 KLOC
Function points	UFP = 153, VAF = 1.03, FP = 157.6
Effort (Basic COCOMO, organic)	12.37 person-months
Development time	6.50 months
Average staffing	1.90 persons
Estimated cost (at ₹25,000 / PM)	₹3,09,227

9. Viva Questions and Answers

Q1. Why is Basic COCOMO called “basic”?

Ans. It uses only program size (KLOC) and project mode to estimate effort, without cost drivers. Intermediate and Detailed COCOMO add 15 cost drivers and phase-level detail.

Q2. What are the three COCOMO modes?

Ans. Organic, semi-detached and embedded, differing in team experience, complexity and constraints.

Q3. Define function point and VAF.

Ans. A function point is a unit of functionality delivered to the user. VAF = 0.65 + 0.01 × ΣF_i adjusts the unadjusted count (UFP) for 14 general system characteristics.

Q4. Give one advantage of FP over LOC.

Ans. FP is independent of programming language and can be estimated from requirements, before code exists.

Q5. What is the three-point estimate?

Ans. Expected size = (Optimistic + 4 × Most likely + Pessimistic) / 6, which weights the most likely value.

Q6. How did you obtain 12.37 person-months?

Ans. E = 2.4 × (4.77)^1.05 using the organic mode and the bottom-up size of 4.77 KLOC.

Q7. Why does the FP-based size differ from the LOC estimate?

Ans. LOC per FP is an average from many projects; our single-file prototype is smaller and denser than that average, and the LOC estimate is specific to our modules.

EXPERIMENT 4

Risk Analysis

Lab week	Week 6
Course outcome	CO2
Tools used	Microsoft Excel
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To identify the risks of Campus Connect and prepare a risk register, probability–impact matrix, risk exposure values and a mitigation plan.

2. Objectives

- Identify project, technical and business risks.
- Quantify each risk using probability and impact.
- Rank risks by risk exposure (RE) and plot them on a risk matrix.
- Prepare mitigation and contingency actions (RMMM).

3. Theory

Risk is an uncertain event that, if it happens, harms the project. Pressman describes **RMMM** – Risk Mitigation, Monitoring and Management. Risks are grouped as *project* (schedule, people, resources), *technical* (design, technology, interfaces) and *business* (market, funding, adoption). The **risk exposure** is RE = P × C, where P is the probability of occurrence and C is the cost or impact if it occurs. Here impact is rated on a 1–5 scale (1 = negligible, 5 = critical) and probability from 0.1 to 0.9. Risks with high RE receive attention first. A **risk matrix** plots probability against impact so that high-priority risks are visible at a glance.

4. Software / Tools Required

- Microsoft Excel for the register and calculations.

5. Procedure

1. Brainstorm risks using a checklist covering people, requirements, technology, schedule and business.
2. Assign probability and impact to every risk.
3. Compute RE = P × impact and rank.
4. Place risks on the 3 × 3 probability–impact matrix.
5. Define mitigation (before) and contingency (after) actions.

6. Application to Campus Connect

6.1 Risk register

ID	Risk	Category	P	Impact	RE
R1	Browser data cleared or unavailable on another device (localStorage only)	Technical	0.7	4	2.8
R2	Requirement creep (chat, payments, uploads requested late)	Project	0.6	3	1.8
R3	Team member unavailable (exams, leave, illness)	Project	0.5	4	2.0
R4	Weak security: passwords kept in plain text in browser storage	Technical	0.8	4	3.2
R5	Layout or behaviour differs across browsers and screen sizes	Technical	0.4	2	0.8
R6	Single large HTML file becomes hard to maintain	Technical	0.5	3	1.5
R7	Spam or fake listings and opportunities (no moderation)	Business	0.5	3	1.5
R8	Low adoption by students and recruiters	Business	0.4	4	1.6
R9	Estimates are inaccurate and schedule slips	Project	0.5	3	1.5
R10	Merge conflicts when four members edit the same file	Project	0.6	2	1.2
	Total risk exposure				**17.9**

Table 4.1 Risk register with probability (P), impact (1–5) and risk exposure (RE = P × Impact)

6.2 Ranking by risk exposure

Rank	ID	RE	Short description
1	R4	3.2	Plain-text passwords
2	R1	2.8	Browser-only data storage
3	R3	2.0	Team member unavailable
4	R2	1.8	Requirement creep
5	R8	1.6	Low adoption
6	R6	1.5	Single-file maintainability
7	R7	1.5	Spam / fake posts
8	R9	1.5	Estimation / schedule slip
9	R10	1.2	Merge conflicts
10	R5	0.8	Cross-browser differences

6.3 Probability–impact matrix

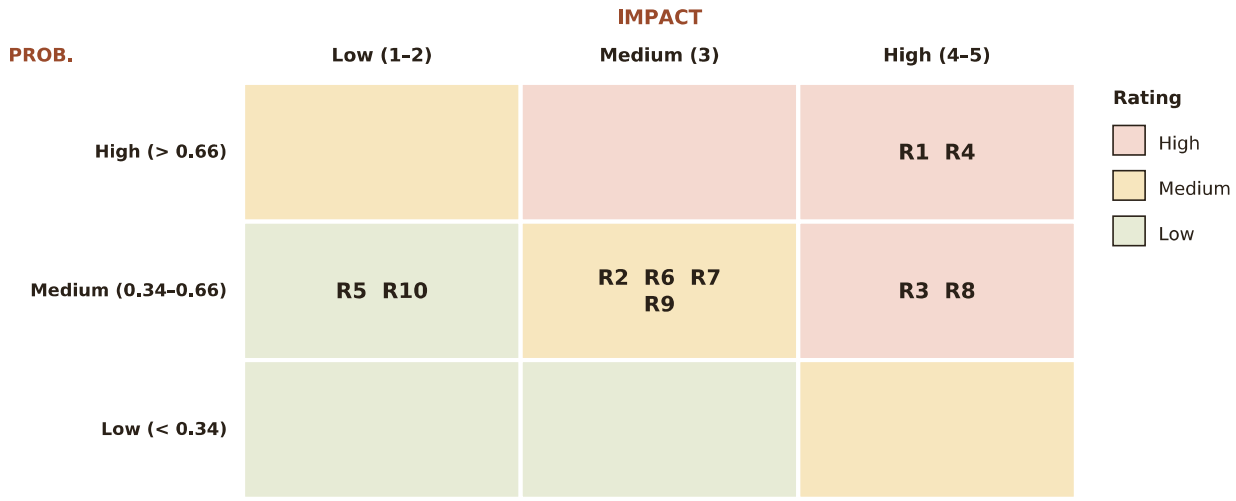


Fig. 4.1 Risk matrix. Probability bands: Low < 0.34, Medium 0.34–0.66, High > 0.66. Impact bands: Low 1–2, Medium 3, High 4–5.

6.4 Mitigation and contingency plan

ID	Mitigation (reduce P or impact)	Contingency (if it occurs)
R1	Show a clear notice; add export / import of data; plan server database in next phase	Re-create data from seed set
R2	Freeze scope in SRS; log extras as future work	Defer to next phase via change request
R3	Pair modules; share code on GitHub; keep tasks small	Re-assign tasks; lead reviews backlog weekly
R4	State limitation in documentation; avoid real passwords; hash on server later	Disable sample real-user data in demo
R5	Test on Chrome, Edge, Firefox and a phone; use responsive CSS	Fix with browser-specific CSS
R6	Group code by module with comments; peer review; consider splitting files	Refactor into separate files
R7	Validate inputs; allow owners to remove posts; plan reporting tool	Manual clean-up of data
R8	Gather feedback from classmates; keep UI simple	Improve features based on feedback
R9	Track weekly progress; re-estimate after each increment	Drop low-priority features
R10	Feature branches; small commits; pull before push	Resolve conflicts with lead; revert if needed

6.5 Risk information sheet for the top risk (R4)

Risk ID / name	R4 – Plain-text passwords in browser storage
Description	The prototype keeps accounts in browser localStorage, which any script or person with access to the browser profile can read.
Probability / impact	0.8 / 4 → RE = 3.2 (High)
Refinement	The cause is the absence of a server and secure authentication. It affects registration and login.
Mitigation	Use only dummy credentials in demonstrations; document the limitation; escape all displayed input; design server-side hashing for the next phase.
Monitoring	Review in every peer review (Experiment 14); check again before the final demonstration.
Management	If sensitive data is entered by a test user, clear browser storage and rotate the sample data.

7. Observations

- Total risk exposure is 17.9; the top three risks (R4, R1, R3) contribute 8.0, i.e. about 45% of it.
- Most high-exposure risks come from the prototype's browser-only design (R1, R4), which confirms why the next phase adds a backend and database.
- Two risks (R4, R1) fall in the High–High cell and need action first.

8. Result

A ten-item risk register with probability, impact and exposure, a 3 × 3 risk matrix and a mitigation / contingency plan was prepared for Campus Connect.

9. Viva Questions and Answers

Q1. What is risk exposure?

Ans. RE = probability × impact; it ranks risks so the most damaging and likely ones are handled first.

Q2. Difference between mitigation and contingency?

Ans. Mitigation actions reduce the chance or effect of a risk before it occurs; contingency actions are executed after it occurs.

Q3. What does RMMM stand for?

Ans. Risk Mitigation, Monitoring and Management.

Q4. Name the types of risk.

Ans. Project risks, technical risks and business risks (also known and predictable, and unpredictable risks).

Q5. Which risk is highest in Campus Connect and why?

Ans. R4, plain-text passwords in browser storage (RE = 3.2), because probability is high and a security failure damages user trust.

Q6. How does a risk matrix help?

Ans. It visualises probability against impact so that the team focuses on the high–high cell first.

Q7. Is risk analysis a one-time activity?

Ans. No. Risks are monitored continuously and the register is updated at the end of every increment.

EXPERIMENT 5

Software Requirement Specification (SRS)

Lab week	Week 7
Course outcome	CO3
Tools used	MS Word, Draw.io
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To prepare a complete IEEE-style Software Requirement Specification for Campus Connect including functional and non-functional requirements, constraints, assumptions and use cases.

2. Objectives

- Elicit and organise requirements of the marketplace and career modules.
- Write uniquely numbered, verifiable functional requirements.
- Specify non-functional requirements, constraints and assumptions.
- Describe key use cases in detail.

3. Theory

An **SRS** is the agreed description of what a system must do. IEEE Std 830-1998 (replaced by ISO/IEC/IEEE 29148) recommends sections for introduction, overall description and specific requirements. A good requirement is *correct, unambiguous, complete, consistent, ranked, verifiable, modifiable* and *traceable*. Functional requirements describe services; non-functional requirements describe quality attributes such as performance, security and usability; constraints restrict design choices.

4. Software / Tools Required

- Microsoft Word (IEEE template), Draw.io for diagrams.

5. Procedure

1. Identify stakeholders and gather needs (project vision, classmates, guide feedback).
2. Write introduction and overall description.
3. Number functional requirements by module and assign priority (H / M / L).
4. Specify non-functional requirements, constraints and assumptions.
5. Write use-case descriptions and review the SRS.

6. Application to Campus Connect

SRS – Campus Connect (IEEE format)

1 Introduction

1.1 Purpose	This SRS specifies the requirements of Campus Connect, a web application that unifies a campus second-hand marketplace and a career portal. Intended readers: developers, testers, the course instructor.
1.2 Scope	The product lets students buy, sell and save items, and lets recruiters publish opportunities that students apply to. It provides accounts, dashboards, notifications and a light / dark theme. The current deliverable is a single-file prototype with browser-based storage.
1.3 Definitions	Listing – an item posted for sale. Opportunity – an internship, full-time role, capstone project or mentorship. Application – a student's request for an opportunity. Status – Submitted, Shortlisted, Interview or Rejected.
1.4 References	IEEE Std 830-1998; Pressman, *Software Engineering: A Practitioner's Approach*; Sommerville, *Software Engineering*; Campus Connect project presentation.
1.5 Overview	Section 2 gives the overall description; Section 3 lists specific requirements; Section 4 gives use cases.

2 Overall description

2.1 Product perspective	Stand-alone browser application with shared navigation. Data is stored in the browser; a future version will use a central server.
2.2 Product functions	Accounts and profiles; marketplace; career portal; dashboard; notifications; theme preference.
2.3 User classes	**Student** – browses, lists and saves items, contacts sellers, searches and applies to opportunities. **Recruiter** – posts and manages opportunities, reviews applicants, updates status.
2.4 Operating environment	Modern web browsers (Chrome, Edge, Firefox, Safari) on desktop and mobile; no installation.
2.5 Design constraints	Single HTML file with embedded CSS and JavaScript; no server-side database; data limited to browser storage capacity.
2.6 Assumptions	Users have a modern browser and JavaScript enabled; sample data is for demonstration; one browser profile represents one user environment.

3 Specific requirements

3.1 Functional requirements (priority H / M / L)

ID	Requirement	Pri.
FR-A1	The system shall allow a visitor to register with name, email, password and role (student or recruiter).	H
FR-A2	The system shall reject registration with an existing email or missing / invalid fields and show a message.	H
FR-A3	The system shall allow registered users to log in and log out.	H
FR-A4	The system shall let users maintain a profile: name, education or company affiliation and skills.	M
FR-M1	The system shall display marketplace listings as cards with title, category, condition, price and seller.	H
FR-M2	The system shall let users search listings by keyword and filter by category and condition.	H
FR-M3	The system shall filter listings by maximum price and sort by newest, price low-to-high and high-to-low.	H
FR-M4	A logged-in user shall create a listing with title, category, condition, price and description.	H
FR-M5	A user shall edit and delete only listings that he or she owns.	H
FR-M6	A user shall save an item as a favourite and remove it from favourites.	M
FR-M7	A user shall send an enquiry to a seller; the seller shall be notified.	H
FR-C1	The system shall list opportunities of types Internship, Full-time, Capstone project and Mentorship.	H
FR-C2	The system shall search opportunities by role, company, skills and type.	H
FR-C3	A recruiter shall post, edit and remove opportunities.	H
FR-C4	A student shall apply to an opportunity with a short note, once per opportunity.	H
FR-C5	A recruiter shall view applications and candidate notes for his or her opportunities.	H
FR-C6	A recruiter shall set application status to Submitted, Shortlisted, Interview or Rejected; the student shall be notified.	H
FR-C7	A student shall view the status of each application.	H
FR-D1	The dashboard shall show my listings, saved items and applications; for recruiters, counts of roles, applicants and shortlisted candidates.	M
FR-D2	The system shall show notifications for enquiries and application updates and allow marking them as read.	M
FR-D3	The system shall provide a light / dark theme and remember the choice.	L

3.2 Non-functional requirements

ID	Attribute	Requirement
NFR-1	Usability	A first-time user shall be able to find items or opportunities and apply within five minutes without help; labels and messages shall be clear.
NFR-2	Performance	Pages, searches and filters shall respond within one second for up to 500 records on a standard laptop.
NFR-3	Reliability	Saved data shall persist across page reloads and browser restarts (same browser profile).
NFR-4	Security	User input shall be validated and escaped before display. Prototype limitation: credentials are held in local browser storage and not suitable for real passwords.
NFR-5	Portability / responsiveness	The interface shall work on screen widths from 360 px to 1920 px in current Chrome, Edge, Firefox and Safari.
NFR-6	Accessibility	Form controls shall have labels; colour contrast shall be adequate in both themes; feedback shall be given for every action.
NFR-7	Maintainability	Code shall be organised by module with comments so that new features can be added without breaking existing ones.

3.3 Design constraints

(C1) Delivery as a single HTML file with embedded CSS and JavaScript. (C2) No server-side database, file upload, payment or real-time chat in this release. (C3) Data limited to the browser storage quota. (C4) Development tools restricted to free software.

3.4 External interfaces

User interface: responsive pages with shared navigation, cards, forms and a theme toggle. **Hardware:** none. **Software:** web browser. **Communication:** none in this release (stand-alone).

3.5 Data requirements

Entities: User, Listing, Enquiry, Saved item, Opportunity, Application, Notification (see class diagram in Experiment 7).

4 Use cases

UC-01 Register / Login

Actor	Visitor (student or recruiter)
Pre-condition	User is not logged in.
Main flow	1. User opens Register or Login. 2. Enters details. 3. System validates. 4. System opens the dashboard.
Alternate flow	Invalid or duplicate details → error message; user corrects and retries.
Post-condition	User session is active.

UC-05 Create a listing

Actor	Student (seller)
Pre-condition	User is logged in.
Main flow	1. Seller opens My Listings and selects Create. 2. Enters title, category, condition, price, description. 3. System validates and saves. 4. Listing appears in the marketplace and on the dashboard.
Alternate flow	Missing title or invalid price → system highlights the field and does not save.
Post-condition	New listing stored.

UC-09 Apply for an opportunity

Actor	Student
Pre-condition	Student is logged in and has not applied to this opportunity.
Main flow	1. Student searches and opens an opportunity. 2. Writes a short note and presses Apply. 3. System saves the application as Submitted. 4. Recruiter is notified.
Alternate flow	Already applied or user is a recruiter → system shows a message and blocks the action.
Post-condition	Application visible on the student's dashboard.

UC-11 Update application status

Actor	Recruiter
Pre-condition	Recruiter owns the opportunity and an application exists.
Main flow	1. Recruiter opens applicants. 2. Reads the note. 3. Selects Shortlisted, Interview or Rejected. 4. System saves the status and notifies the student.
Alternate flow	Invalid transition (for example Rejected → Shortlisted) → system refuses.
Post-condition	Status updated.

4.1 Requirement-to-use-case traceability

Use case	Requirements covered
UC-01	FR-A1, FR-A2, FR-A3
UC-05	FR-M4, FR-M5, FR-D1
UC-09	FR-C2, FR-C4, FR-D2
UC-11	FR-C5, FR-C6, FR-C7, FR-D2

7. Observations

- 21 functional requirements were grouped into four areas (accounts, marketplace, career, dashboard) and seven non-functional requirements were defined.
- Each requirement has a unique ID, so test cases in Experiment 10 can be traced to it.
- Security and persistence are the weakest areas of the prototype and are documented as constraints.

8. Result

A complete IEEE-format SRS for Campus Connect was prepared, with 21 functional requirements, 7 non-functional requirements, constraints, assumptions and four detailed use cases.

9. Viva Questions and Answers

Q1. What is an SRS and who uses it?

Ans. The SRS is the contract-like description of what the software must do. Customers, designers, developers, testers and managers all use it.

Q2. List the characteristics of a good SRS.

Ans. Correct, unambiguous, complete, consistent, ranked for importance, verifiable, modifiable and traceable.

Q3. Functional vs non-functional requirements?

Ans. Functional requirements state what the system does (e.g. apply to an opportunity). Non-functional requirements state how well it does it (speed, security, usability).

Q4. Give an example of a constraint in our project.

Ans. The system must be delivered as a single HTML file with browser-based storage and no server.

Q5. What is a use case?

Ans. A description of how an actor interacts with the system to achieve a goal, with main and alternate flows.

Q6. Why is traceability important?

Ans. It links requirements to design and tests so that every requirement is implemented and verified, and change impact is easy to assess.

Q7. Which IEEE standard describes the SRS?

Ans. IEEE Std 830-1998, now superseded by ISO/IEC/IEEE 29148:2018.

EXPERIMENT 6

UML Modeling

Lab week	Week 8
Course outcome	CO3
Tools used	StarUML / Visual Paradigm, Draw.io
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To model the behaviour of Campus Connect using a Use Case Diagram, Activity Diagram, Sequence Diagram and Collaboration (Communication) Diagram.

2. Objectives

- Identify actors and use cases from the SRS.
- Model workflow with an activity diagram using swimlanes.
- Model object interaction over time with a sequence diagram.
- Show the same interaction as a collaboration diagram.

3. Theory

The **Unified Modeling Language (UML)** is a standard notation for visualising, specifying and documenting software. Behavioural diagrams used here are:

Diagram	Purpose	Main elements
Use case	Shows who uses the system and for what.	Actors, use-case ellipses, system boundary, association, include / extend.
Activity	Shows the flow of work or control.	Start / end nodes, actions, decisions, swimlanes, flows.
Sequence	Shows message order between objects over time.	Lifelines, synchronous / return messages, time axis.
Collaboration (communication)	Shows objects, their links and numbered messages.	Objects, links, numbered messages.

4. Software / Tools Required

- StarUML or Visual Paradigm Community Edition (CASE tools); Draw.io as an alternative.

5. Procedure

1. Read the SRS and list actors (Student, Recruiter) and goals.
2. Draw the use case diagram with a system boundary.
3. Select a key workflow (apply for an opportunity and review it) and draw the activity diagram with swimlanes.
4. Draw the sequence diagram for the same workflow using lifelines for UI, logic and storage.
5. Select another interaction (send an enquiry) and draw the collaboration diagram with numbered messages.
6. Check each diagram against the SRS for consistency.

6. Application to Campus Connect

6.1 Use case diagram

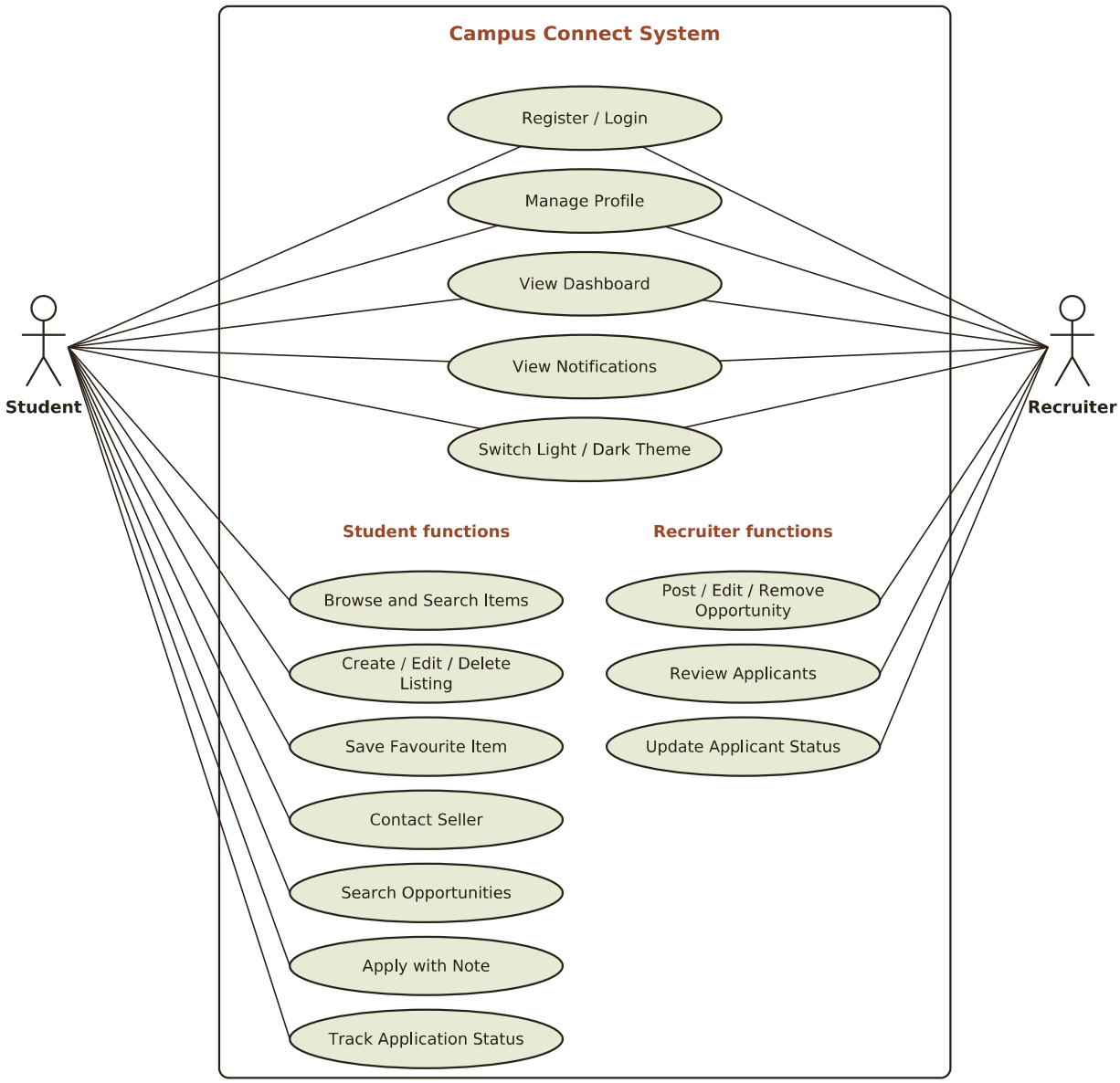


Fig. 6.1 Use case diagram. Register / Login is required («include») before any use case that changes data.

Actor	Description	Main use cases
Student	Campus user who buys, sells, saves items, searches and applies for opportunities.	Browse items, manage listings, save items, contact seller, search opportunities, apply, track status
Recruiter	Company or campus representative who publishes opportunities.	Post / edit / remove opportunities, review applicants, update status
Both	Any registered user.	Register / login, manage profile, view dashboard and notifications, switch theme

6.2 Activity diagram – apply for an opportunity and review it

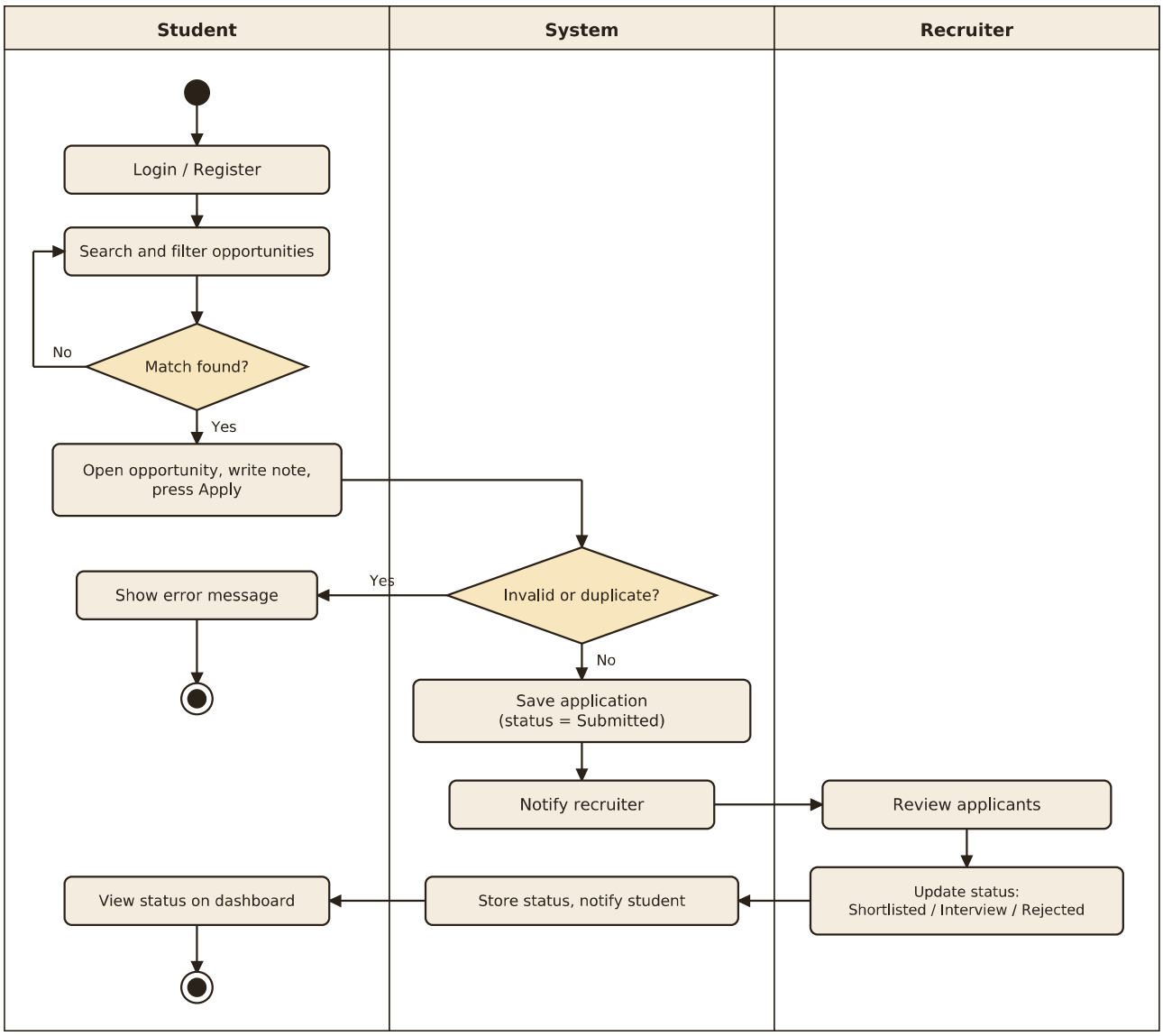


Fig. 6.2 Activity diagram with swimlanes for Student, System and Recruiter

6.3 Sequence diagram – apply and update status

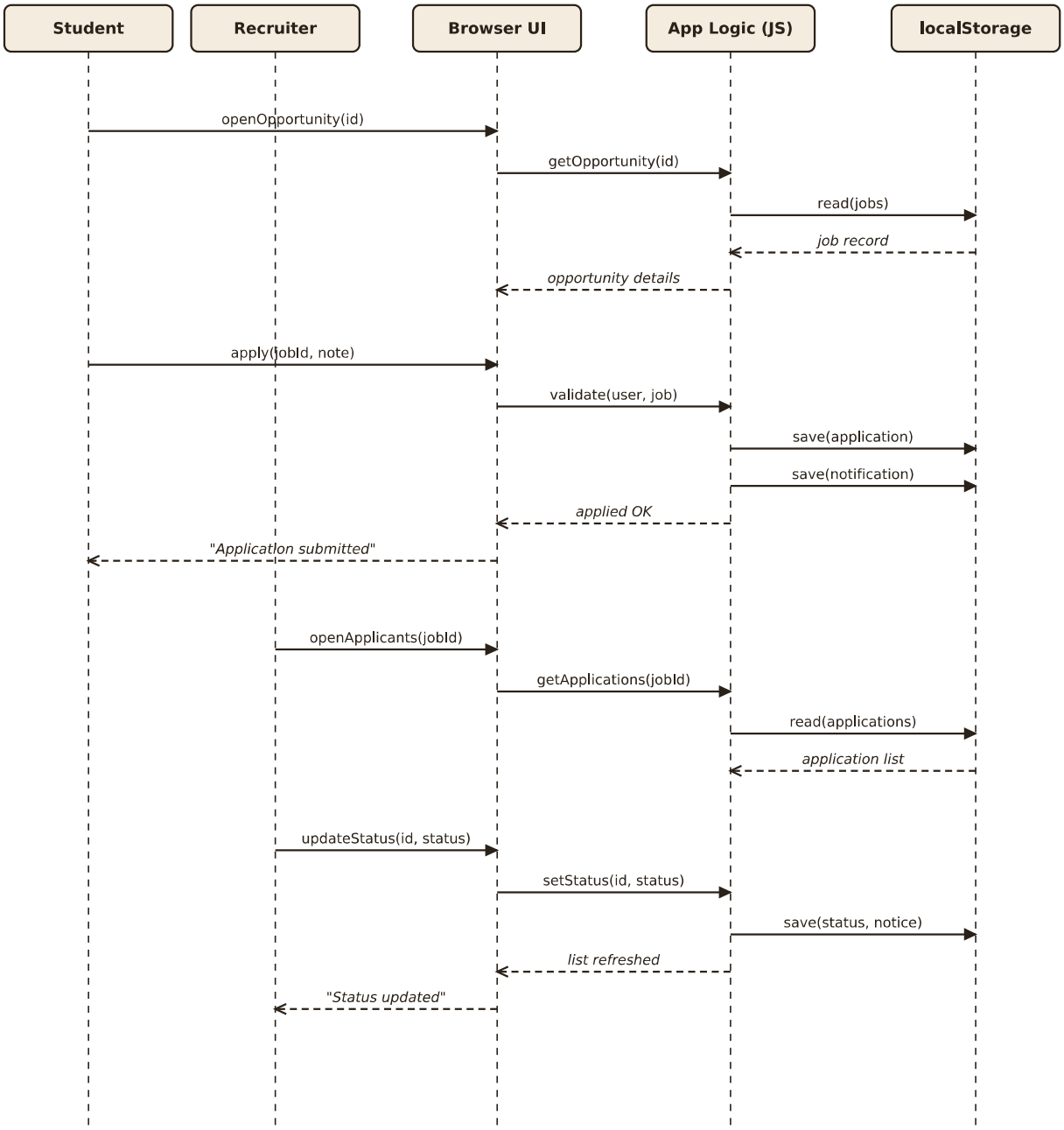
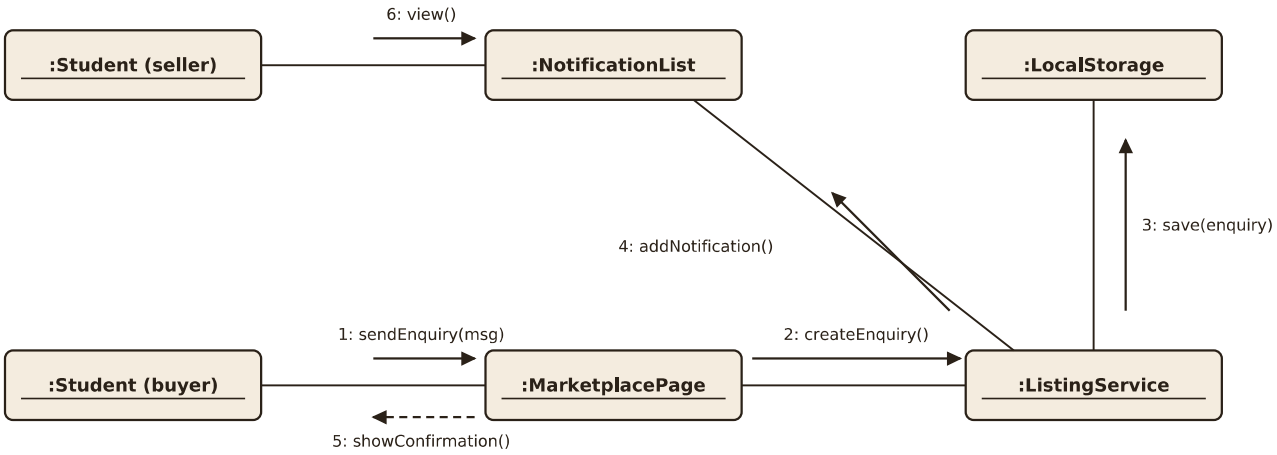


Fig. 6.3 Sequence diagram. Solid arrows are calls, dashed arrows are returns.

6.4 Collaboration diagram – send an enquiry to a seller



Numbers show message order

Fig. 6.4 Collaboration diagram for "Contact seller"

7. Observations

- Two actors and 15 use cases cover all functional requirements of the SRS.
- The activity diagram shows the system lane validating duplicates, which supports FR-C4.
- Storage is accessed only through the logic layer in the sequence diagram, matching the layered architecture of Experiment 8.
- Sequence and collaboration diagrams carry the same information; the sequence diagram emphasises time, the collaboration diagram emphasises links.

8. Result

Use case, activity, sequence and collaboration diagrams for Campus Connect were developed and found consistent with the SRS.

9. Viva Questions and Answers

Q1. What is an actor?

Ans. An external role (person or system) that interacts with the system, e.g. Student and Recruiter.

Q2. Difference between include and extend?

Ans. «include» means a use case always uses another (Apply includes Login). «extend» adds optional behaviour under a condition.

Q3. What are swimlanes in an activity diagram?

Ans. Partitions that show which actor or component performs each action.

Q4. How is a sequence diagram different from a collaboration diagram?

Ans. Sequence diagrams order messages along a vertical time axis; collaboration diagrams show the same messages on object links using sequence numbers.

Q5. What do dashed arrows represent in a sequence diagram?

Ans. Return messages (replies) from the called object.

Q6. Name the types of UML diagrams.

Ans. Structural: class, object, component, package, deployment, composite structure. Behavioural: use case, activity, state machine, sequence, communication, interaction overview, timing.

Q7. Why model before coding?

Ans. Models expose misunderstandings early, communicate the design to the team and form documentation.

EXPERIMENT 7

Design Modeling

Lab week	Week 9
Course outcome	CO3
Tools used	StarUML, Visual Paradigm
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To prepare the structural design of Campus Connect with Class, Object, Package and Component diagrams.

2. Objectives

- Identify classes, attributes, operations and relationships from the SRS.
- Create an object diagram showing sample instances.
- Organise the system into packages.
- Identify software components and dependencies.

3. Theory

A **class diagram** shows classes with attributes and operations and the relationships association, aggregation, composition, generalisation and dependency. Multiplicities (1, *, 0..1) state how many objects participate. An **object diagram** is a snapshot of instances at a moment in time. A **package diagram** groups related elements and shows dependencies between groups. A **component diagram** shows replaceable modules with their provided and required interfaces.

4. Software / Tools Required

- StarUML / Visual Paradigm Community Edition.

5. Procedure

1. Extract nouns from the SRS as candidate classes and verbs as operations.
2. Define attributes and visibility; remove duplicates.
3. Establish relationships and multiplicities; use generalisation for Student and Recruiter.
4. Instantiate selected classes to draw an object diagram.
5. Group classes into packages and check dependency direction.
6. Identify components and their interfaces.

6. Application to Campus Connect

6.1 Class diagram

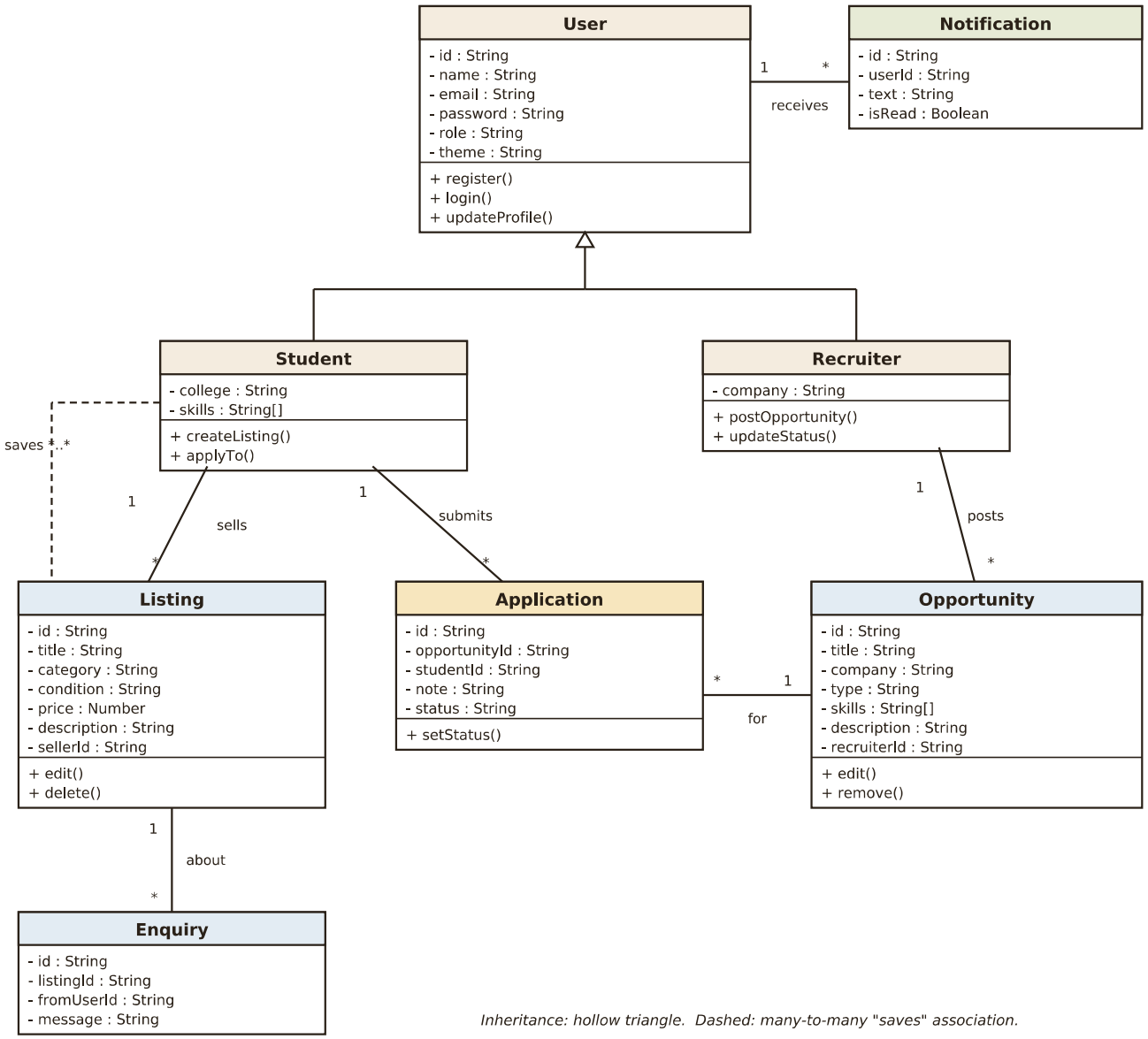


Fig. 7.1 Class diagram of Campus Connect

Relationship	Meaning	Multiplicity
Student – Listing (sells)	A student owns many listings; each listing has one seller.	1 : *
Student – Listing (saves)	Students save many listings; a listing can be saved by many.	* : *
Listing – Enquiry	A listing receives many enquiries.	1 : *
Recruiter – Opportunity (posts)	A recruiter posts many opportunities.	1 : *
Student – Application (submits)	A student submits many applications.	1 : *
Opportunity – Application	An opportunity receives many applications; each application is for one opportunity.	1 : *
User – Notification	A user receives many notifications.	1 : *
Student, Recruiter → User	Generalisation: both inherit id, name, email, password, role, theme.	inheritance

6.2 Object diagram

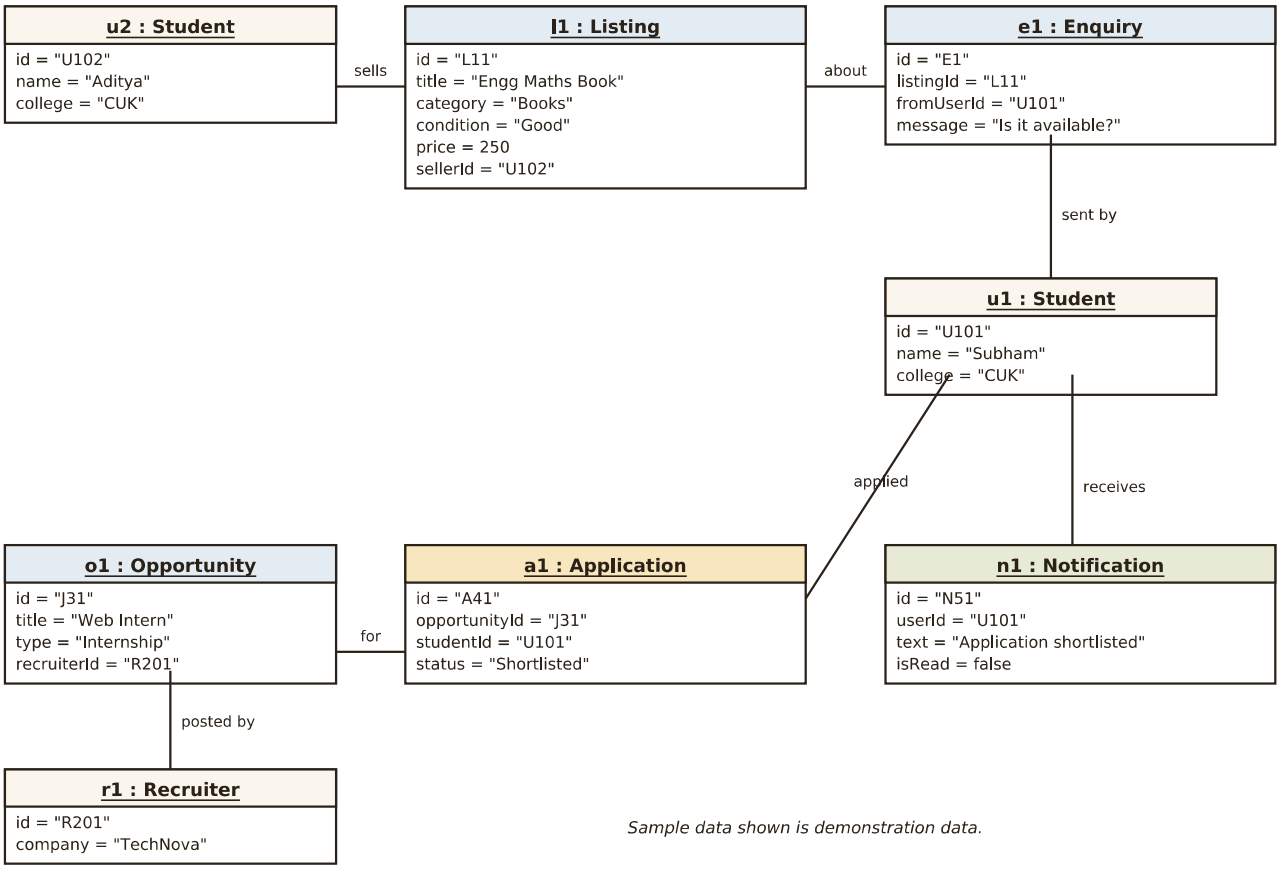


Fig. 7.2 Object diagram: one marketplace enquiry and one job application with sample values

6.3 Package diagram

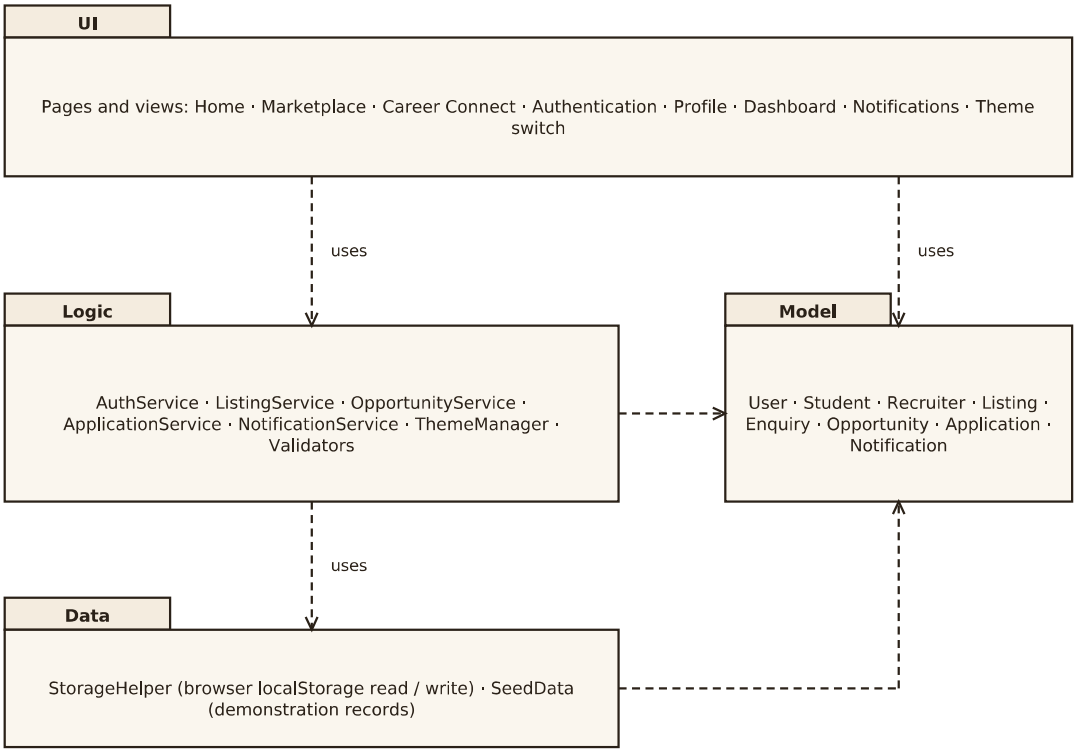
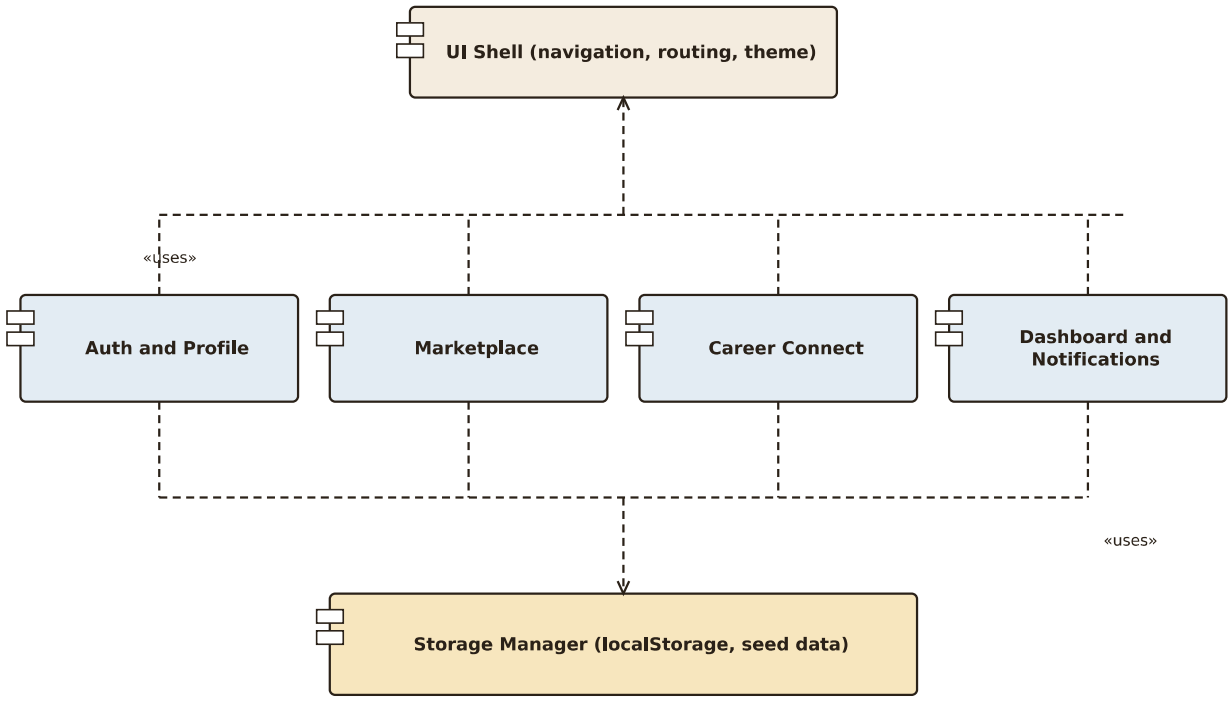


Fig. 7.3 Package diagram showing layered dependencies

6.4 Component diagram



Enquiries and application updates raise notifications through the Dashboard and Notifications component.

Fig. 7.4 Component diagram

Component	Responsibility	Used by
UI Shell	Navigation, routing between pages, theme switch.	All pages
Auth and Profile	Register, login, logout, profile data.	UI Shell
Marketplace	Listings, search, filters, sorting, favourites, enquiries.	UI Shell, Dashboard
Career Connect	Opportunities, search, applications, status.	UI Shell, Dashboard
Dashboard and Notifications	Personal summaries, alerts.	UI Shell
Storage Manager	Read / write JSON in localStorage; seed data.	All logic components

7. Observations

- Eight domain classes were identified; Student and Recruiter specialise User, so common attributes are defined once.
- Application links Student and Opportunity, which resolves the many-to-many relationship between them.
- Dependencies in the package diagram point downward only (UI → Logic → Data), avoiding cycles.

8. Result

Class, object, package and component diagrams were designed and are consistent with the SRS and the behavioural models.

9. Viva Questions and Answers

Q1. What is the difference between aggregation and composition?

Ans. Both are whole–part relationships. In composition the part cannot exist without the whole (strong ownership); in aggregation the part can exist independently.

Q2. Why use generalisation for Student and Recruiter?

Ans. They share user attributes and operations; inheritance avoids duplication and models the “is-a” relationship.

Q3. How does an object diagram differ from a class diagram?

Ans. A class diagram shows the general structure; an object diagram shows specific instances with actual attribute values at one moment.

Q4. What is a multiplicity?

Ans. The number of instances that can take part in a relationship, such as 1, 0..1 or *.

Q5. Purpose of a package diagram?

Ans. To group related classes and show dependencies between groups, supporting modular design.

Q6. What is a component?

Ans. A modular, replaceable part of a system that encapsulates implementation and exposes interfaces.

Q7. How did you find the classes?

Ans. By analysing nouns in the SRS (user, listing, enquiry, opportunity, application, notification) and checking that each has attributes and behaviour.

EXPERIMENT 8

Architecture Design

Lab week	Week 10
Course outcome	CO3
Tools used	StarUML, Draw.io
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To design the layered architecture, client–server architecture and deployment diagram of Campus Connect.

2. Objectives

- Select an architectural style for the prototype and justify it.
- Model the layers and their responsibilities.
- Compare the current stand-alone design with a proposed client–server design.
- Draw a deployment diagram for the current and proposed systems.

3. Theory

Software architecture is the high-level structure of a system: its components, their relationships and the rules governing them. In a **layered architecture** each layer uses services of the layer below and offers services to the layer above, which improves modifiability and testing. A **client-server architecture** separates service requesters (clients) from providers (servers) over a network. A **deployment diagram** shows physical nodes (devices, servers), execution environments and the artefacts deployed on them.

4. Software / Tools Required

- StarUML / Draw.io.

5. Procedure

1. List quality goals (modifiability, usability, portability, testability).
2. Choose the architectural style and define the layers.
3. Describe the interfaces between layers.
4. Model the proposed client–server evolution.
5. Draw the deployment diagram.

6. Application to Campus Connect

6.1 Layered architecture

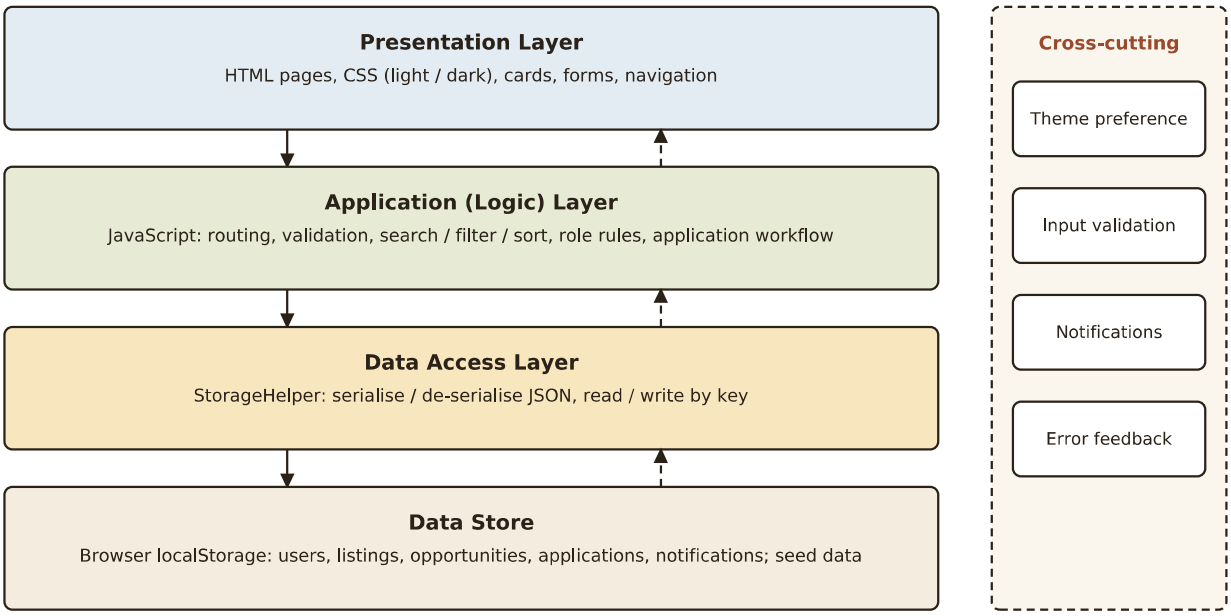


Fig. 8.1 Layered architecture of the prototype

Layer	Responsibility	Example elements
Presentation	Show pages, collect input, give feedback; theme and responsive layout.	Home, marketplace, careers, dashboard, forms
Application (logic)	Validate input; apply rules (ownership, one application per opportunity); search, filter, sort; status workflow.	Filter and sort functions, apply, update status
Data access	Save and load records as JSON by key.	StorageHelper
Data store	Persist data in the browser; supply demonstration data.	localStorage, seed data

6.2 Client–server architecture

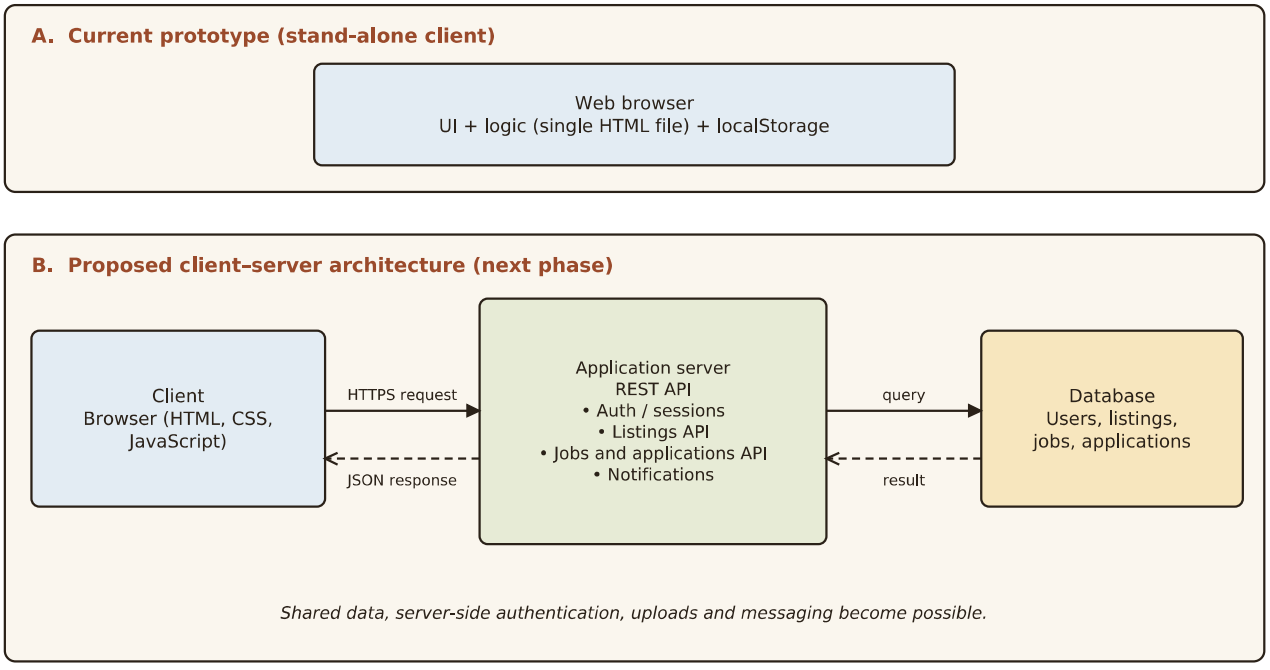


Fig. 8.2 Current stand-alone design and proposed client–server design

6.3 Deployment diagram

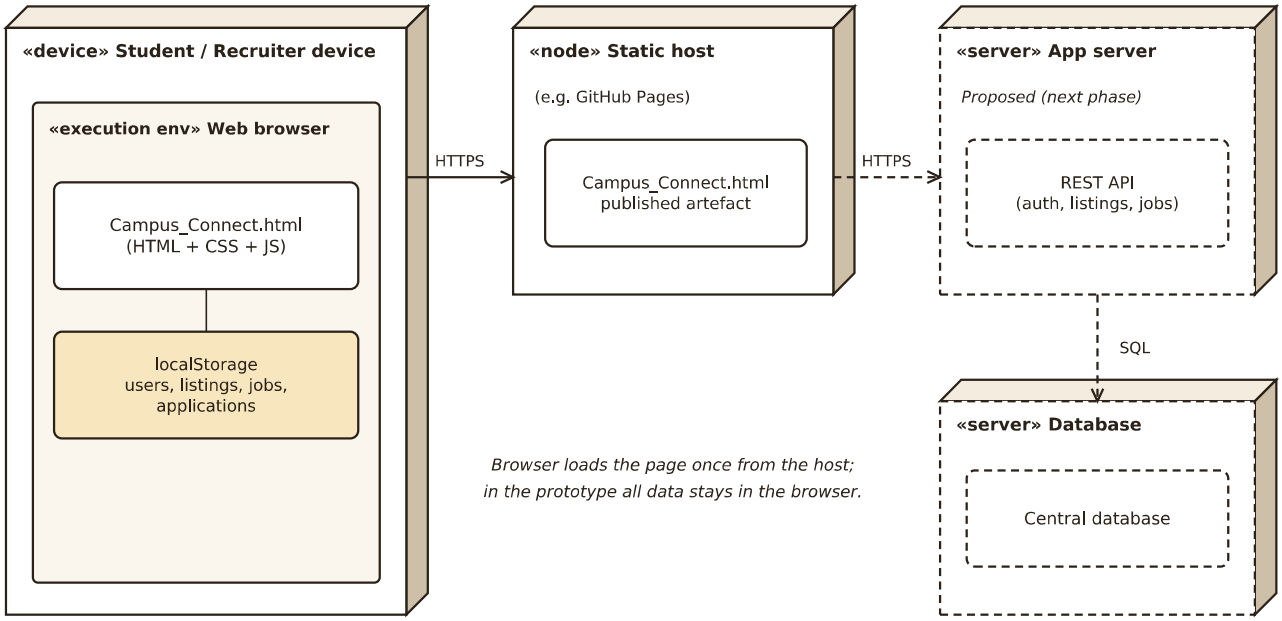


Fig. 8.3 Deployment diagram (dashed nodes are proposed)

6.4 Architectural decisions

Decision	Rationale	Trade-off
Layered style	Separates UI, rules and storage; easy to test and change.	Slight overhead of calls between layers.
Single-file HTML prototype	Zero setup, easy demonstration, matches lab scope.	Harder to maintain as code grows; no shared data.
Browser localStorage	No server needed; persists between sessions.	Not shared, limited size, not secure for credentials.
Storage behind one helper	Storage can be replaced by an API call later without changing the logic layer.	Requires discipline to avoid direct access.
Proposed REST backend	Enables shared data, secure authentication, uploads and messaging.	More infrastructure and security work.

7. Observations

- Layering isolates all storage access in one helper, so moving to a database mainly changes the data-access layer.
- The deployment diagram shows that the prototype can be hosted on any static host, since all logic runs in the browser.
- Cross-cutting concerns (theme, validation, notifications) are shared by all layers.

8. Result

Layered, client–server and deployment designs were prepared for Campus Connect, along with a justified evolution path from prototype to campus-ready platform.

9. Viva Questions and Answers

Q1. What is software architecture?

Ans. The fundamental organisation of a system: its components, relationships and design principles.

Q2. Why choose a layered architecture?

Ans. It separates concerns, simplifies testing and lets one layer change (for example storage) with little effect on others.

Q3. Difference between a 2-tier and 3-tier client–server system?

Ans. In 2-tier the client talks directly to the database server; in 3-tier an application server sits between client and database.

Q4. What is shown in a deployment diagram?

Ans. Hardware nodes, execution environments and the software artefacts deployed on them with their connections.

Q5. What are the limitations of the current architecture?

Ans. No shared data between users, weak security, limited storage and no server-side verification, uploads or messaging.

Q6. What are cross-cutting concerns?

Ans. Concerns used by many layers such as validation, logging, theme and notifications.

Q7. How does the proposed architecture improve the system?

Ans. A central server and database allow shared accounts and listings, secure authentication, uploads, moderation and real-time features.

EXPERIMENT 9

Interface Design

Lab week	Week 11
Course outcome	CO3
Tools used	Figma, Balsamiq (low-fidelity wireframes)
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To design the Login screen, Dashboard, Reports (applicant report) and navigation flow of Campus Connect.

2. Objectives

- Apply user interface design principles and usability heuristics.
- Create low-fidelity wireframes for key screens.
- Define the navigation flow between screens.
- Evaluate the design against Nielsen's heuristics.

3. Theory

Good interface design makes the system easy to learn and efficient to use. Shneiderman's **eight golden rules** include consistency, shortcuts, informative feedback, closure in dialogs, error prevention, easy reversal of actions, user control and reduced short-term memory load. Nielsen's **ten usability heuristics** provide a checklist for evaluation. **Wireframes** are low-fidelity layouts that show structure and content without visual styling; **prototypes** add interaction.

4. Software / Tools Required

- Figma (free plan) or Balsamiq for wireframes and clickable prototype.
- The wireframes in this record are drawn in a Balsamiq-like low-fidelity style.

5. Procedure

1. List screens from the use cases and decide the layout of the shared navigation bar.
2. Define design guidelines (colours, typography, spacing, theme).
3. Sketch wireframes of Login / Register, Dashboard and Recruiter report.
4. Define the navigation flow between screens.
5. Evaluate with the heuristic checklist and refine.

6. Application to Campus Connect

6.1 Design guidelines

Element	Decision
Look and feel	Warm ivory background with earthy accent colours, soft borders and clean cards, minimalist layout.
Themes	Light and dark mode with a toggle in the navigation bar; the choice is remembered.
Navigation	One shared top bar (Home, Marketplace, Careers, Alerts, Profile) on every page.
Layout	Responsive grid of cards that stacks on small screens; forms are single column.
Feedback	Message after every action (saved, deleted, applied, error); empty-state text when no results.
Accessibility	Labels for inputs, visible focus, adequate contrast, status shown by text and colour.

6.2 Login and Register screens

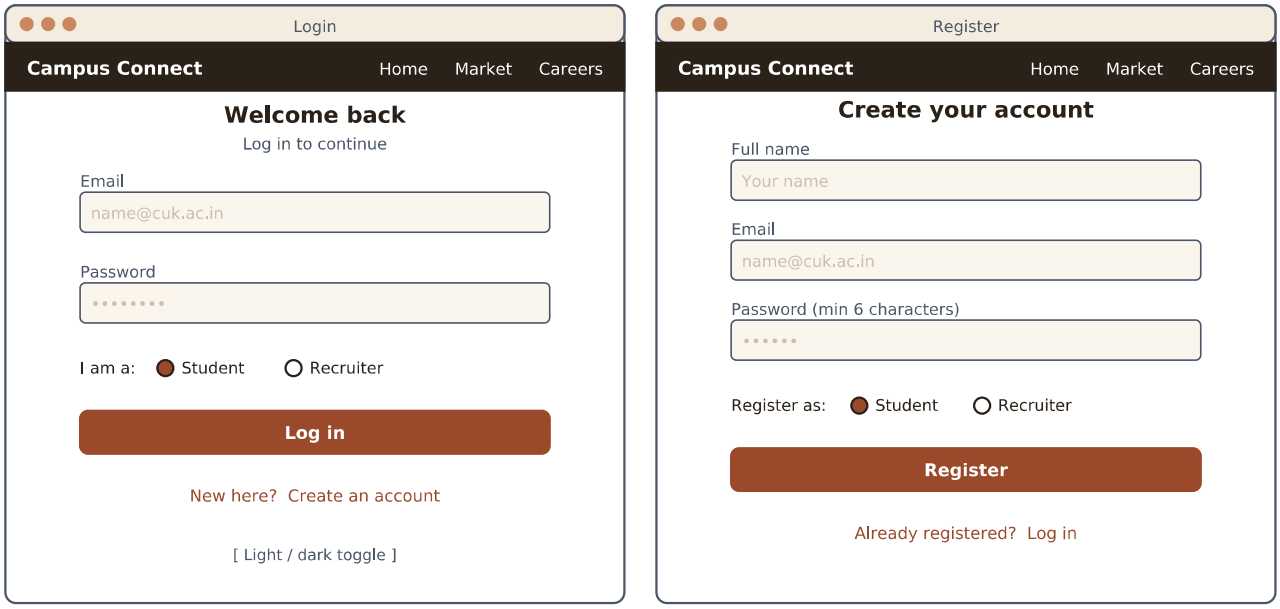


Fig. 9.1 Wireframes of the Login and Register screens

6.3 Dashboard

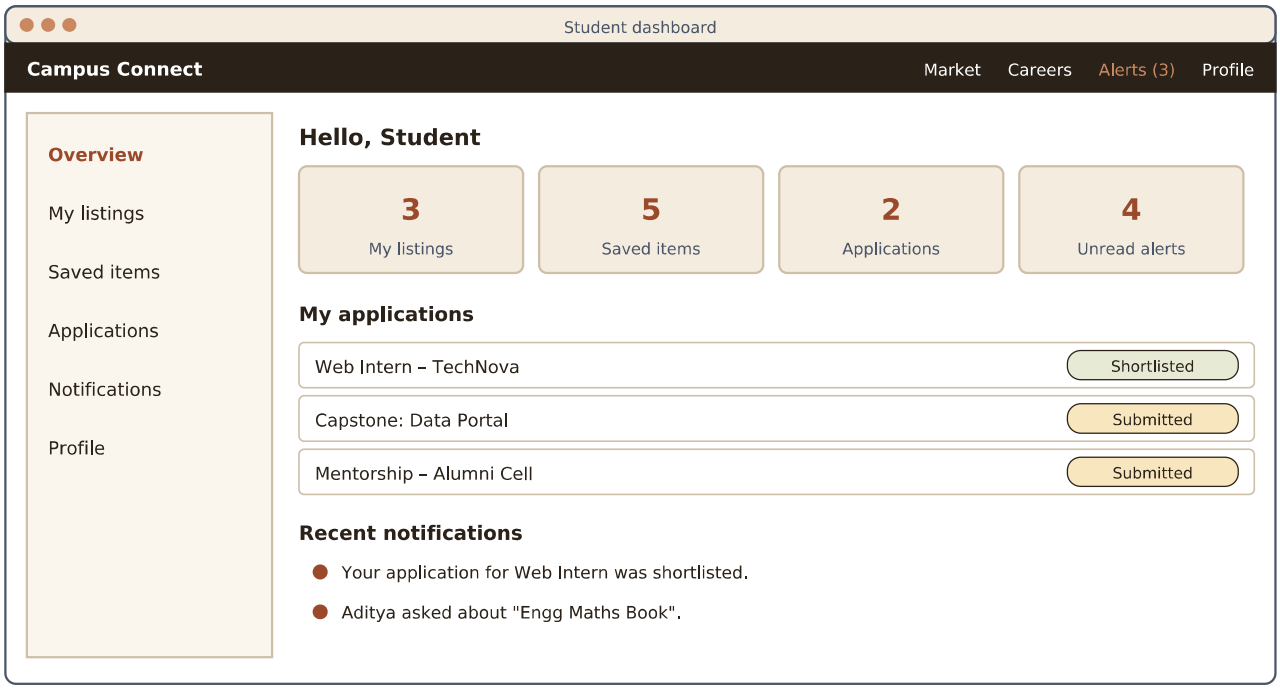


Fig. 9.2 Student dashboard wireframe: summary cards, applications and notifications

6.4 Reports – recruiter dashboard and applicant report

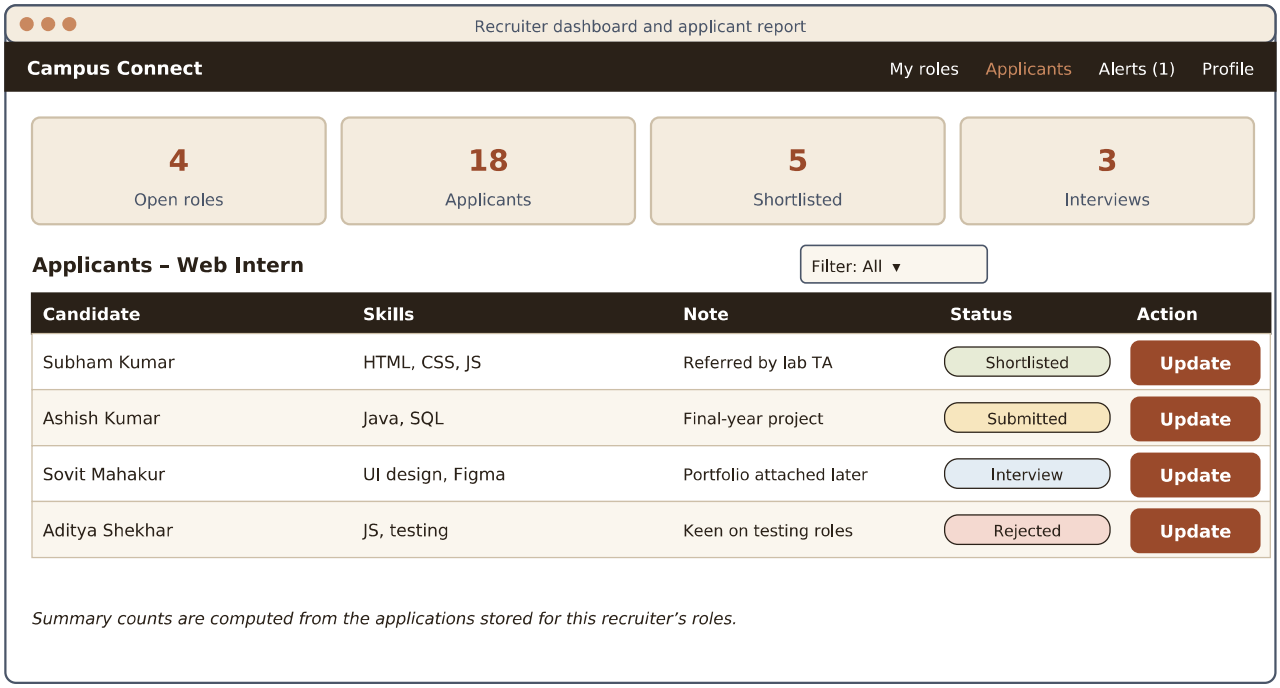


Fig. 9.3 Recruiter report wireframe (sample names are demonstration data)

6.5 Navigation flow

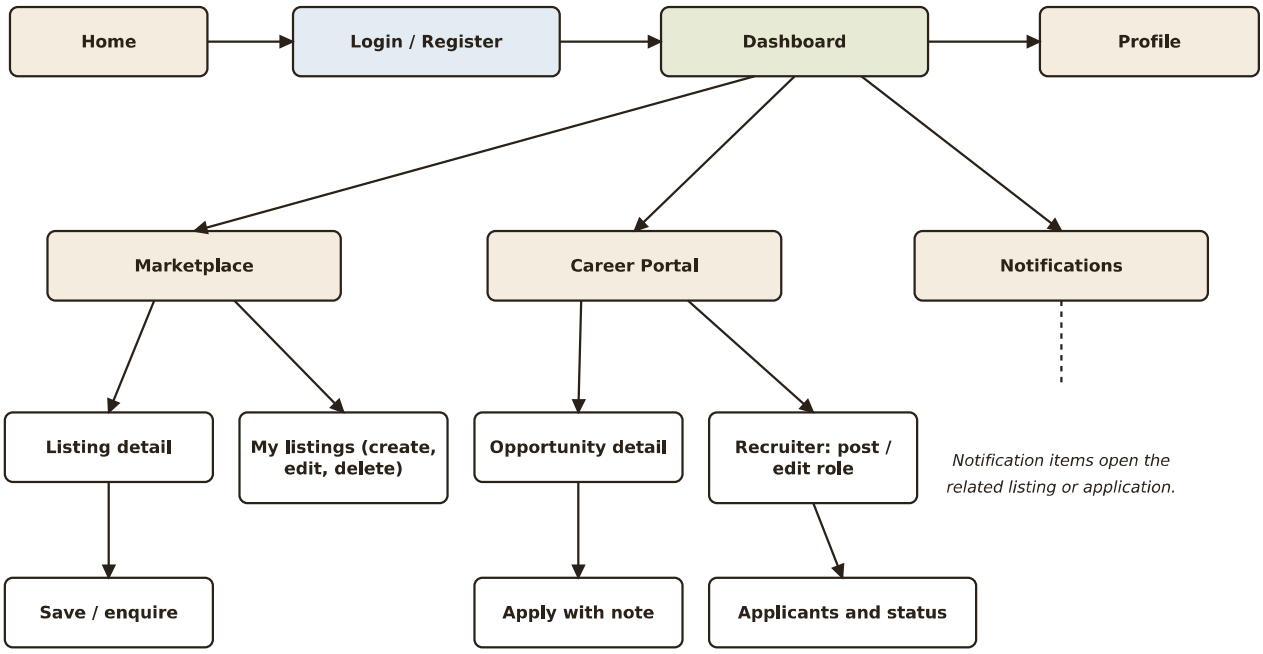


Fig. 9.4 Navigation flow between screens

6.6 Heuristic evaluation

Heuristic	How the design meets it
Visibility of system status	Status chips (Submitted, Shortlisted, Interview, Rejected), unread alert count, confirmation messages.
Match with real world	Familiar terms: listing, condition, price, apply, shortlisted.
User control and freedom	Edit and delete own listings; remove favourites; back via navigation bar.
Consistency and standards	Same navigation, card style and button placement across modules.
Error prevention	Validation on price and required fields; one application per opportunity; confirmation before delete.
Recognition rather than recall	Visible filters and categories; dashboard shows all personal items.
Flexibility and efficiency	Search plus filters and sorting; dashboard shortcuts.
Aesthetic and minimalist design	Clean cards, one primary action per card, generous spacing.
Help users recover from errors	Plain-language error messages next to the field.
Help and documentation	Short hints on forms; empty-state messages explain what to do next.

7. Observations

- The same navigation bar on every page keeps the two modules feeling like one product.
- Dashboard cards give a quick overview, while detail lists appear below.
- Status chips use both colour and text, supporting accessibility.

8. Result

Wireframes for Login / Register, Dashboard and Reports and a navigation flow were designed and checked against usability heuristics. They can be rebuilt as a clickable prototype in Figma using the same layout.

9. Viva Questions and Answers

Q1. What is a wireframe?

Ans. A low-fidelity layout of a screen that shows structure and content placement without detailed styling.

Q2. Name some of Shneiderman’s golden rules.

Ans. Strive for consistency, enable shortcuts, offer informative feedback, design dialogs with closure, prevent errors, permit easy reversal, support internal locus of control and reduce short-term memory load.

Q3. What is Nielsen’s heuristic “visibility of system status”?

Ans. The system should always inform users about what is going on through timely feedback, e.g. application status chips.

Q4. Why support a dark theme?

Ans. It reduces eye strain in low light, saves battery on some displays and is a common user expectation.

Q5. Why use the same navigation on every page?

Ans. Consistency lowers learning effort and helps users know where they are.

Q6. Difference between low-fidelity and high-fidelity prototypes?

Ans. Low-fidelity prototypes are quick sketches for structure; high-fidelity prototypes look and behave like the final product.

Q7. How do you make the interface accessible?

Ans. Label inputs, keep colour contrast adequate, do not rely on colour alone, and give clear focus and feedback.

EXPERIMENT 10

Test Planning

Lab week	Week 12
Course outcome	CO4
Tools used	Excel, JUnit / PyTest (for automated tests in later phase)
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To prepare a test plan, test scenarios, test cases and test data for Campus Connect.

2. Objectives

- Define scope, approach and criteria of testing.
- Derive test scenarios from the SRS.
- Write detailed test cases with expected results.
- Prepare test data and a requirement-to-test traceability matrix.

3. Theory

A **test plan** (IEEE 829 / ISO 29119) describes what will be tested, how, by whom, when and with what pass / fail criteria. A **test scenario** is a situation to verify; a **test case** specifies input, pre-conditions, steps and expected output. Testing levels include unit, integration, system and acceptance testing. **Black-box** techniques use the specification; **white-box** techniques use the code structure. Verification asks “are we building the product right?” and validation asks “are we building the right product?”.

4. Software / Tools Required

- Excel for test case sheets; browsers Chrome, Edge and Firefox; JUnit / PyTest for unit tests in a later phase.

5. Procedure

1. Define test items, features and approach.
2. Derive scenarios for each functional area.
3. Write test cases with ID, inputs, steps and expected result.
4. Prepare test data.
5. Map test cases to requirements.

6. Application to Campus Connect

6.1 Test plan

Test plan ID	TP-CC-01
Introduction	Plan for verifying the Campus Connect prototype against SRS version 1.0.
Test items	Campus_Connect.html – accounts, marketplace, career portal, dashboard, notifications, theme.
Features to be tested	FR-A1 to FR-D3 and NFR-1, 3, 5, 6 (see SRS).
Features not tested	Server-side features, real verification, uploads, payments, real-time chat (outside scope).
Approach	System testing by black-box techniques (equivalence partitioning, boundary value, decision table, state transition – Experiment 12); white-box testing of the filter logic (Experiment 11); exploratory testing of responsive layout and themes.
Pass / fail criteria	A test passes when actual result equals expected result. Release requires 100% of high-priority tests passed and no open high-severity defects.
Suspension / resumption	Suspend if the page fails to load or data storage is unavailable; resume after the fix is verified.
Test deliverables	Test plan, scenarios, test cases, test data, defect log, test summary.
Environment	Windows 11 / Android phone; Chrome, Edge, Firefox (latest); screen widths 360, 768, 1366 and 1920 px; browser storage enabled.
Schedule	Weeks 12–14 of the project schedule.
Risks	Browser data cleared during tests (R1); schedule slip (R9). Reset sample data before each cycle.
Responsibilities	Test lead: Aditya Shekhar; testers: all members; reviewer: Dr. Rohit Kumar B.

Responsibility split is a suggestion; edit it to match your team.

6.2 Test scenarios

ID	Scenario	Requirement
TS-01	A visitor registers and logs in as student and as recruiter.	FR-A1–A3
TS-02	A user maintains profile information.	FR-A4
TS-03	A user searches, filters and sorts marketplace listings.	FR-M1–M3
TS-04	A seller creates, edits and deletes own listings.	FR-M4, M5
TS-05	A user saves items and contacts a seller.	FR-M6, M7
TS-06	A user searches opportunities by role, company, skills and type.	FR-C1, C2
TS-07	A recruiter posts, edits and removes opportunities.	FR-C3
TS-08	A student applies and tracks status; a recruiter reviews and updates it.	FR-C4–C7
TS-09	Dashboard and notifications show correct data.	FR-D1, D2
TS-10	Theme toggle and responsive layout work.	FR-D3, NFR-5

6.3 Test cases

TC	Scenario	Input / steps	Expected result	Type
TC-01	TS-01	Register: name "Test User", valid email, password of 8 characters, role Student.	Account created; user logged in; dashboard opens.	+
TC-02	TS-01	Register again with the same email.	Error: email already registered; no new account.	−
TC-03	TS-01	Register with empty name or invalid email format.	Field highlighted; message shown; no account created.	−
TC-04	TS-01	Login with registered email and wrong password.	Error message; user stays logged out.	−
TC-05	TS-01	Login with correct credentials, then log out.	Dashboard opens; after logout, protected pages ask to log in.	+
TC-06	TS-02	Edit profile: add skills "JS, SQL"; save; reload.	Skills saved and still shown after reload.	+
TC-07	TS-03	Search "book" in marketplace.	Only listings whose title / text matches "book" are shown.	+
TC-08	TS-03	Filter category Books, condition Good, max price 300.	Only items matching all three conditions are shown.	+
TC-09	TS-03	Sort by price low-to-high, then high-to-low.	Order changes ascending then descending.	+
TC-10	TS-03	Search for a word that matches nothing.	Empty-state message shown; no error.	−
TC-11	TS-04	Create listing "Study Table", Furniture, Good, price 1800.	Listing appears in marketplace and My Listings.	+
TC-12	TS-04	Create listing with price −1 or text "abc".	Validation error; listing not saved.	−
TC-13	TS-04	Edit own listing price to 1500; delete another own listing.	Price updated; deleted listing disappears.	+
TC-14	TS-04	Try to edit a listing owned by another user.	Edit / delete options are not available.	−
TC-15	TS-05	Save an item, open Saved Items, remove it.	Item appears then is removed.	+
TC-16	TS-05	Send enquiry "Is it available?" to a seller; log in as seller.	Seller sees a notification with the enquiry.	+
TC-17	TS-06	Search opportunities with skill "JavaScript" and type Internship.	Only matching internships listed.	+
TC-18	TS-07	Recruiter posts "Web Intern"; edits title; removes another role.	Role appears, title changes, removed role is gone.	+
TC-19	TS-08	Student applies with note; applies again to the same role.	First: status Submitted. Second: blocked with message.	+ / −
TC-20	TS-08	Recruiter sets Submitted → Shortlisted → Interview.	Status changes; student notified each time.	+
TC-21	TS-08	Recruiter tries to set Rejected → Shortlisted.	Change refused.	−
TC-22	TS-09	Recruiter dashboard after the steps above.	Counts of roles, applicants and shortlisted are correct.	+
TC-23	TS-10	Toggle dark mode; reload page.	Theme changes and persists.	+
TC-24	TS-10	Open at 360 px width.	Cards stack; no horizontal scrolling; text readable.	+

Table 10.1 Test cases (+ positive, − negative)

6.4 Test data

Data set	Values
Users	Student: student1@test.in / Test@1234; Student 2: student2@test.in; Recruiter: recruiter1@test.in (company TechNova).
Listings	Engg Maths Book (Books, Good, 250); Study Table (Furniture, Good, 1800); Casio Calculator (Electronics, Good, 600); Python Book (Books, Fair, 150); Physics Notes (Books, Good, 100).
Opportunities	Web Intern (Internship, JS / HTML); Data Portal (Capstone project, Python); Alumni Mentorship (Mentorship); Software Engineer (Full-time, Java).
Boundary data	Price: 0, 1, 50000, 50001; password lengths 5, 6, 20, 21.
Invalid data	Email without @; empty title; price "abc"; very long note (301 characters).

6.5 Requirement traceability

Requirement	Test cases
FR-A1–A3	TC-01 to TC-05
FR-A4	TC-06
FR-M1–M3	TC-07 to TC-10
FR-M4, M5	TC-11 to TC-14
FR-M6, M7	TC-15, TC-16
FR-C1, C2	TC-17
FR-C3	TC-18
FR-C4–C7	TC-19 to TC-21
FR-D1, D2	TC-16, TC-22
FR-D3, NFR-5	TC-23, TC-24

7. Observations

- 24 test cases cover all 21 functional requirements; negative cases are included for validation and access rules.
- Boundary and invalid data sets are prepared for the black-box experiment that follows.
- Environment diversity (three browsers, four widths) addresses the cross-browser risk R5.

8. Result

A test plan, ten test scenarios, 24 test cases, test data and a traceability matrix were prepared for Campus Connect.

9. Viva Questions and Answers

Q1. What is the difference between a test scenario and a test case?

Ans. A scenario is a high-level situation to test; a test case gives exact inputs, steps and expected results for it.

Q2. Define verification and validation.

Ans. Verification checks that the product is built correctly against specifications; validation checks that it meets the user's real needs.

Q3. What are the levels of testing?

Ans. Unit, integration, system and acceptance testing.

Q4. What are pass / fail and suspension criteria?

Ans. Pass / fail criteria decide whether a test or release is acceptable; suspension criteria define when testing stops, such as a blocker defect.

Q5. Why include negative test cases?

Ans. They check that the system rejects invalid input and unauthorised actions gracefully.

Q6. What is a traceability matrix?

Ans. A table linking requirements to test cases to show that every requirement is tested.

Q7. Difference between black-box and white-box testing?

Ans. Black-box tests the behaviour against specifications without looking at code; white-box uses knowledge of the internal code structure.

EXPERIMENT 11

White Box Testing

Lab week	Week 13
Course outcome	CO4
Tools used	Visual Studio Code / Eclipse, browser developer console
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To perform white-box testing of the Campus Connect marketplace filter logic using cyclomatic complexity, basis path testing, statement coverage and decision coverage.

2. Objectives

- Draw the flow graph of a function and compute cyclomatic complexity.
- Derive a basis set of independent paths and design a test case for each.
- Design test cases that achieve 100% statement and decision coverage.
- Interpret complexity for maintainability.

3. Theory

White-box (glass-box) testing designs tests from the internal structure of the code. A **flow graph** represents a program with nodes (statements or blocks) and edges (control flow). **Cyclomatic complexity** (McCabe) is V(G) = E − N + 2 = P + 1 = number of regions, where E is edges, N nodes and P predicate nodes. It equals the number of linearly independent paths. **Basis path testing** executes every independent path at least once. **Statement coverage** executes every statement; **decision (branch) coverage** makes every decision evaluate both true and false. Decision coverage subsumes statement coverage.

V(G)	Risk / interpretation
1 – 10	Simple procedure, low risk
11 – 20	Moderate complexity, moderate risk
21 – 50	Complex, high risk
> 50	Untestable, very high risk

4. Software / Tools Required

- Visual Studio Code or Eclipse; browser developer console (or Node.js) to run the function.

5. Procedure

1. Select the function under test and number its statements.
2. Draw the flow graph and identify predicate nodes.
3. Compute V(G) by three methods.
4. List the basis set of independent paths.
5. Design one test case per path with expected output.
6. Design test cases for statement and decision coverage.
7. Execute and record results.

6. Application to Campus Connect

6.1 Function under test

The marketplace search and filter feature (FR-M2, FR-M3) is implemented as the function below. It is a representative version of the filter logic; copy the same analysis onto the function names used in your own Campus_Connect.html.

```
function filterListings(list, category, keyword, maxPrice, sortBy) {
  var result = [];                                        // 1
  for (var i = 0; i < list.length; i++) {                 // 2  D1
    var item = list[i];                                   // 3
    if (item.price > maxPrice) continue;                  // 4  D2
    if (category !== "All") {                             // 5  D3
      if (item.category !== category) continue;           // 6  D4
    }
    if (keyword !== "") {                                 // 7  D5
      if (item.title.toLowerCase()
              .indexOf(keyword.toLowerCase()) === -1)     // 8  D6
        continue;
    }
    result.push(item);                                    // 9
  }                                                       // 10 (i++)
  if (sortBy === "price") {                               // 11 D7
    result.sort(function (a, b) {                         // 12
      return a.price - b.price; });
  }
  return result;                                          // 13
}
```

6.2 Flow graph

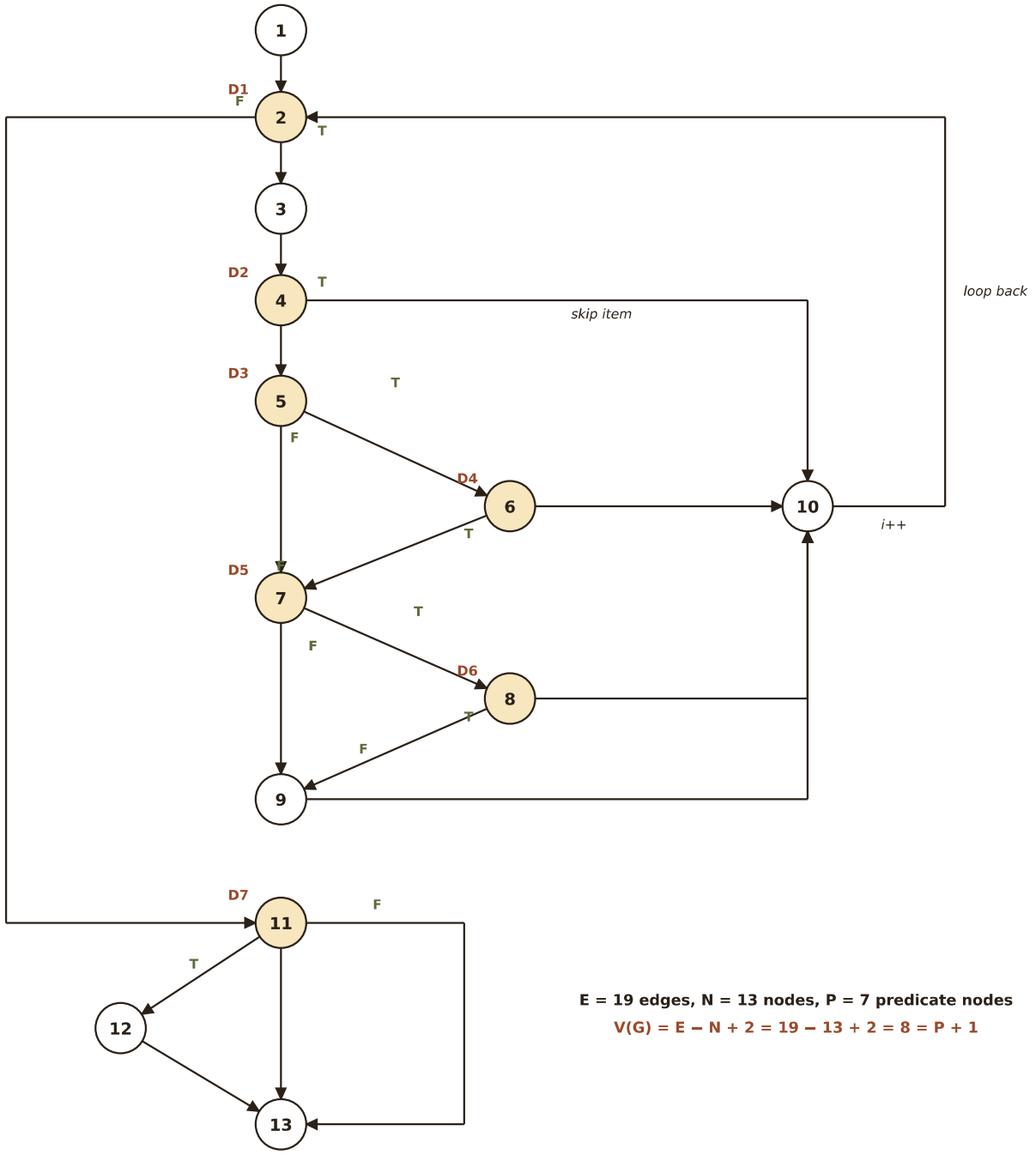


Fig. 11.1 Flow graph of filterListings(). Circles are numbered statements; D1–D7 are decisions.

6.3 Cyclomatic complexity

Method	Calculation	V(G)
Edges and nodes	E − N + 2 = 19 − 13 + 2	8
Predicate nodes	P + 1 = 7 + 1	8
Regions	Count of enclosed regions + outer region	8

V(G) = 8 lies in the 1–10 band, so the function is simple enough to test completely. Eight independent paths must be executed (the independence of the eight paths below was confirmed by computing the rank of their edge-incidence matrix, which is 8).

6.4 Basis set of independent paths

Path	Node sequence	Meaning
P1	1-2-11-13	Empty list; no sorting
P2	1-2-11-12-13	Empty list; sort by price
P3	1-2-3-4-10-2-11-13	Item priced above the limit is skipped
P4	1-2-3-4-5-6-10-2-11-13	Category selected but item has another category → skipped
P5	1-2-3-4-5-7-8-10-2-11-13	Category All; keyword not found in title → skipped
P6	1-2-3-4-5-6-7-9-10-2-11-13	Category matches; no keyword → added
P7	1-2-3-4-5-6-7-8-9-10-2-11-13	Category matches; keyword found → added
P8	1-2-3-4-5-7-9-10-2-11-13	Category All; no keyword → added

6.5 Test cases for basis path testing

Sample data: **L1** = [{Engg Maths Book, Books, 250}, {Study Table, Furniture, 1800}, {Casio Calculator, Electronics, 600}, {Python Book, Books, 150}]. Item = {title, category, price}. maxPrice = 1000 unless stated.

TC	Path	Inputs (list, category, keyword, sortBy)	Expected output	Actual
W1	P1	[], "All", "", "newest"	[]	
W2	P2	[], "All", "", "price"	[]	
W3	P3	[Study Table 1800], "All", "", "newest"	[] (price above 1000)	
W4	P4	[Study Table 800 Furniture], "Books", "", "newest"	[] (wrong category)	
W5	P5	[Engg Maths Book 250], "All", "calc", "newest"	[] (keyword absent)	
W6	P6	[Engg Maths Book 250], "Books", "", "newest"	[Engg Maths Book]	
W7	P7	[Engg Maths Book 250], "Books", "maths", "newest"	[Engg Maths Book]	
W8	P8	[Engg Maths Book 250], "All", "", "newest"	[Engg Maths Book]	

Table 11.1 Basis path test cases. Fill the Actual column after executing the function.

6.6 Statement coverage

A single test executes all 13 statements: **SC-1**: list L2 = L1 + {Physics Notes, Books, 100}, category "Books", keyword "book", maxPrice 1000, sortBy "price". The loop runs for every item (statements 2–10), items pass or fail each check (4, 6, 8), matching items are added (9), and the sort branch runs (11, 12) before return (13). Expected output: [Python Book (150), Engg Maths Book (250)]. **Statement coverage = 13 / 13 = 100%.**

6.7 Decision (branch) coverage

TC	Inputs	Decisions exercised	Expected output
DC-1	L2, "Books", "book", 1000, "price"	D1 T/F; D2 T (Study Table) / F; D3 T; D4 T (Casio Calculator) / F; D5 T; D6 T (Physics Notes) / F; D7 T	[Python Book, Engg Maths Book]
DC-2	L2, "All", "", 1000, "newest"	D3 F; D5 F; D7 F (others repeated)	[Engg Maths Book, Casio Calculator, Python Book, Physics Notes]

Together DC-1 and DC-2 make every decision evaluate both ways: **14 / 14 branch outcomes = 100% decision coverage**. Because compound conditions were split into nested decisions, condition coverage is also complete.

Criterion	Test cases needed	Coverage achieved
Statement coverage	1 (SC-1)	100%
Decision coverage	2 (DC-1, DC-2)	100%
Basis path coverage	8 (W1–W8)	100% of independent paths

7. Observations

- Statement coverage needs only one test but would miss errors in paths that skip items; decision coverage needs two and basis path testing needs eight.

- V(G) = 8 is within the recommended limit of 10, so the function does not need refactoring.
- Nested decisions make each path clear; combining conditions with && would reduce nodes but require condition coverage analysis.

8. Result

The flow graph of the marketplace filter function was drawn, V(G) = 8 was computed, eight independent paths were tested and 100% statement and decision coverage were achieved by the designed test cases.

9. Viva Questions and Answers

Q1. What is cyclomatic complexity and how is it computed?

Ans. McCabe's measure of the number of independent paths: V(G) = E − N + 2, or P + 1, or the number of regions. For our function it is 8.

Q2. What is basis path testing?

Ans. A white-box technique that derives a set of linearly independent paths from the flow graph and tests each one at least once.

Q3. Statement coverage vs decision coverage?

Ans. Statement coverage executes every statement; decision coverage makes each decision true and false. Decision coverage is stronger.

Q4. Can 100% statement coverage miss bugs?

Ans. Yes. A branch that never executes an else path, or a loop never iterated zero times, can hide defects even though every statement ran.

Q5. What does V(G) = 8 indicate?

Ans. The function has moderate size, is easy to test and maintain (limit 10), and needs eight independent test paths.

Q6. What is a predicate node?

Ans. A node with two or more outgoing edges, i.e. a decision such as an if or loop condition.

Q7. Why is white-box testing done by developers?

Ans. It requires knowledge of the code structure and is best performed during unit testing.

EXPERIMENT 12

Black Box Testing

Lab week	Week 12
Course outcome	CO4
Tools used	Excel, browser, JUnit / PyTest
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To test Campus Connect using black-box techniques: Boundary Value Analysis, Equivalence Partitioning, Decision Table testing and State Transition testing.

2. Objectives

- Derive test cases from input domains without using the code.
- Apply BVA and equivalence partitioning to numeric and categorical inputs.
- Use a decision table for business rules and a state diagram for the application workflow.
- Record expected and actual results.

3. Theory

Black-box testing checks functionality against the specification. **Equivalence partitioning** divides input values into classes that the system treats alike, so one value per class represents the class. **Boundary value analysis** adds tests at the edges of ranges, where defects cluster; for a range [min, max] test min−1, min, min+1, a nominal value, max−1, max and max+1. **Decision tables** list combinations of conditions and the resulting actions. **State transition testing** checks valid and invalid changes of state caused by events.

4. Software / Tools Required

- Excel for test tables; Chrome browser for execution; optional JUnit / PyTest scripts.

5. Procedure

1. Identify input fields and rules from the SRS.
2. Partition inputs into valid and invalid classes; choose a representative value for each.
3. Select boundary values.
4. Build the decision table and the state transition table.
5. Execute and record actual results and status.

6. Application to Campus Connect

The input rules used below (price 1–50,000, password 6–20 characters) are the validation rules assumed for the prototype. If your implementation uses different limits, change the boundary values accordingly. The Actual / Status column is left for you to fill after executing each test.

6.1 Boundary Value Analysis

(a) Listing price field – valid range ₹1 to ₹50,000.

TC	Price entered	Class	Expected result	Actual / Status
B1	0	Invalid (min − 1)	Rejected with message	
B2	1	Valid (min)	Accepted	
B3	2	Valid (min + 1)	Accepted	
B4	25000	Valid (nominal)	Accepted	
B5	49999	Valid (max − 1)	Accepted	
B6	50000	Valid (max)	Accepted	

TC	Price entered	Class	Expected result	Actual / Status
B7	50001	Invalid (max + 1)	Rejected with message	

(b) Password length – valid 6 to 20 characters.

TC	Length	Class	Expected result	Actual / Status
B8	5	Invalid	Rejected	
B9	6	Valid (min)	Accepted	
B10	7	Valid	Accepted	
B11	19	Valid	Accepted	
B12	20	Valid (max)	Accepted	
B13	21	Invalid	Rejected	

6.2 Equivalence Partitioning

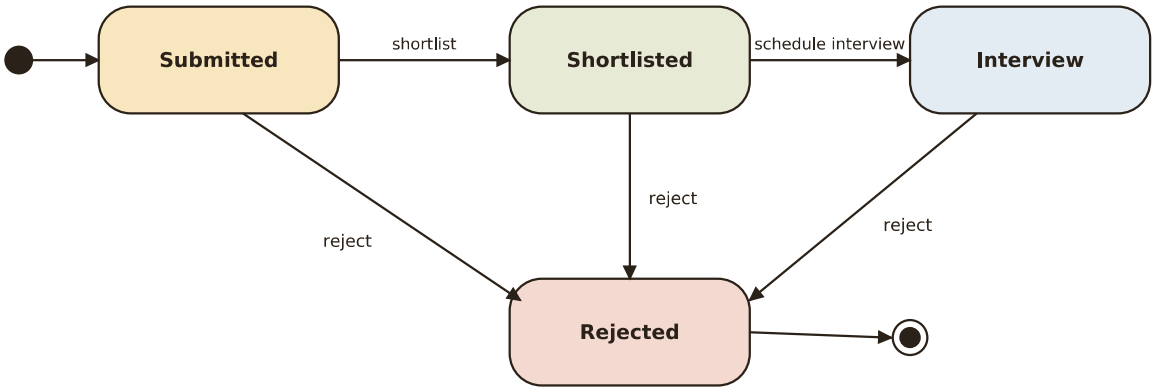
Input	Valid classes	Invalid classes	Representative tests
Listing category	Books, Electronics, Furniture, Cycles, Clothing, Stationery	Empty; value not in list	Books (valid); "" and "Toys" (invalid)
Condition	New, Good, Fair	Empty; unknown value	Good; ""
Email	Contains one @ and a domain with dot	No @; no domain; spaces; empty	a@cuk.ac.in; abc.com; a@b; ""
Role	Student, Recruiter	None selected	Student; none
Max price filter	Positive number	Negative; non-numeric; empty (means no limit)	500; −100; "abc"
Opportunity type	Internship, Full-time, Capstone project, Mentorship	Other / empty	Internship; ""
Application note	1–300 characters	0 characters (if required); > 300	40 chars; 301 chars

6.3 Decision table – applying to an opportunity

Conditions / actions	R1	R2	R3	R4
C1 User is logged in	N	Y	Y	Y
C2 User role is Student	–	N	Y	Y
C3 Not already applied to this opportunity	–	–	N	Y
A1 Save application with status Submitted and notify recruiter				X
A2 Ask the user to log in	X			
A3 Block: recruiters cannot apply		X		
A4 Show “already applied” message			X	

TC	Rule	Input	Expected result	Actual / Status
D1	R1	Visitor not logged in presses Apply	Login prompt shown	
D2	R2	Recruiter opens an opportunity and tries Apply	Action blocked / hidden	
D3	R3	Student who already applied presses Apply	“Already applied” message; no duplicate	
D4	R4	Student applies first time with a note	Status Submitted; recruiter notified	

6.4 State transition testing – application status



Rejected is terminal; Interview ends the tracked workflow in the prototype.

Fig. 12.1 State diagram of an application

Current state	Event	Next state	Valid?
Submitted	Shortlist	Shortlisted	Yes
Submitted	Reject	Rejected	Yes
Shortlisted	Schedule interview	Interview	Yes
Shortlisted	Reject	Rejected	Yes
Interview	Reject	Rejected	Yes
Submitted	Schedule interview	–	No (must shortlist first)
Rejected	Shortlist	–	No (terminal state)
Interview	Shortlist	–	No (cannot go back)

TC	Start state	Event	Expected result	Actual / Status
S1	Submitted	Shortlist	Shortlisted; student notified	
S2	Shortlisted	Schedule interview	Interview; student notified	
S3	Submitted	Reject	Rejected; student notified	
S4	Submitted	Schedule interview	Refused	
S5	Rejected	Shortlist	Refused	
S6	Interview	Shortlist	Refused	

7. Observations

- Boundary tests (B1, B7, B8, B13) are the most likely to expose off-by-one errors in validation.
- The decision table shows four distinct rules, so four test cases cover all rule combinations.
- State testing confirms both allowed transitions and forbidden ones; the latter protect data integrity.
- Total black-box tests designed: 13 boundary + 4 decision + 6 state = 23, plus the equivalence classes in 6.2.

8. Result

Black-box test cases using BVA, equivalence partitioning, decision table and state transition techniques were designed for Campus Connect; results are recorded in the status column when executed.

9. Viva Questions and Answers

Q1. Why do defects cluster at boundaries?

Ans. Developers often use < instead of <= or miscount limits, so boundaries are where off-by-one errors appear.

Q2. How many tests does BVA give for one range?

Ans. Seven values with min−1, min, min+1, nominal, max−1, max, max+1 (or four to five for the basic form).

Q3. What is an equivalence class?

Ans. A set of inputs the system treats the same way; testing one representative is assumed to represent all.

Q4. When is a decision table useful?

Ans. When behaviour depends on combinations of conditions, such as the rules for applying to an opportunity.

Q5. Why test invalid state transitions?

Ans. To confirm the system blocks illegal changes, e.g. re-shortlisting a rejected candidate.

Q6. Difference between black-box and white-box testing?

Ans. Black-box uses specifications and input / output behaviour; white-box uses internal code structure.

Q7. What is the “pesticide paradox”?

Ans. Repeating the same tests stops finding new defects, so test cases must be reviewed and extended regularly.

EXPERIMENT 13

Configuration Management (Git and GitHub)

Lab week	Week 14
Course outcome	CO5
Tools used	Git, GitHub, VS Code
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To manage the source code and documents of Campus Connect with Git and GitHub: installation, repository creation, commit, push, pull, branching, merging and conflict resolution.

2. Objectives

- Install and configure Git; create a local and a remote repository.
- Record changes with commits and share them with push and pull.
- Use branches for features and merge them.
- Resolve a merge conflict.
- Define a branching strategy and release tag.

3. Theory

Software Configuration Management (SCM) identifies, controls and audits changes to software work products. Its main activities are configuration identification, version control, change control, status accounting and audit. **Git** is a distributed version control system: every clone holds the full history. Files move between the *working directory*, the *staging area* (index) and the *repository*. A **branch** is an independent line of development; a **merge** combines branches; a **conflict** occurs when the same lines are changed differently. **GitHub** hosts remote repositories and adds pull requests, issues and code review. A **baseline** is a formally reviewed version that serves as the basis for further work.

4. Software / Tools Required

- Git (command line), GitHub account, Visual Studio Code, web browser.

5. Procedure – commands

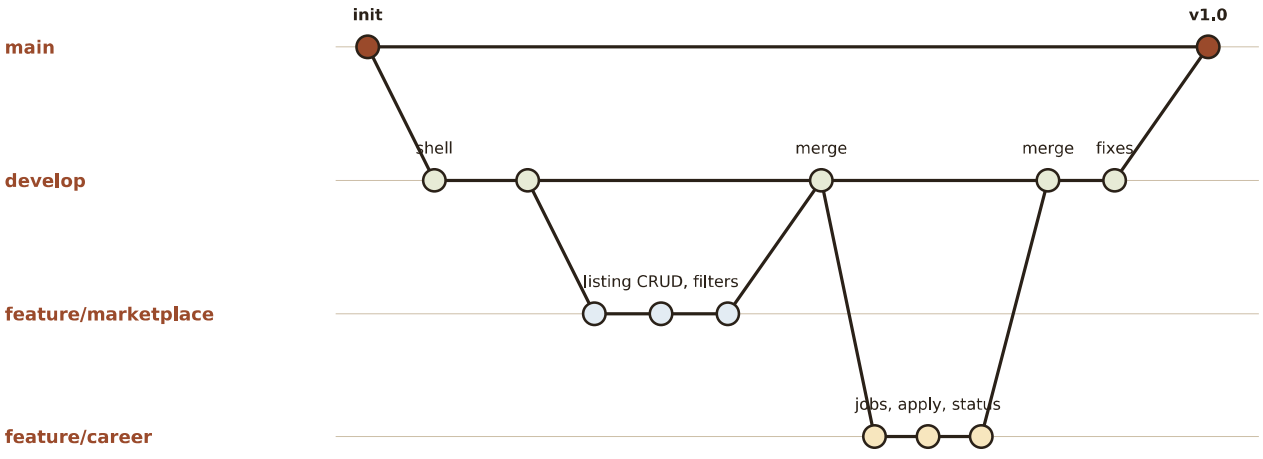
Step	Command	Purpose
1 Install	`git --version`	Verify installation (download from git-scm.com).
2 Configure	`git config --global user.name "Your Name"` `git config --global user.email "you@example.com"`	Set identity for commits.
3 Create repository	`mkdir campus-connect && cd campus-connect` `git init`	Initialise a local repository.
4 Add files	`git add Campus_Connect.html README.md`	Stage files.
5 Commit	`git commit -m "Add initial Campus Connect shell"`	Save a snapshot with a message.
6 Connect remote	`git remote add origin https://github.com/<user>/campus-connect.git` `git branch -M main`	Link to GitHub repository.
7 Push	`git push -u origin main`	Upload commits to GitHub.
8 Pull	`git pull origin main`	Download and merge teammates' changes.
9 Branch	`git checkout -b feature/marketplace`	Create and switch to a feature branch.
10 Merge	`git checkout develop` `git merge feature/marketplace`	Combine a feature into develop.
11 Inspect	`git status` `git log --oneline --graph --all`	View state and history.
12 Tag	`git tag -a v1.0 -m "Final demo release"` `git push origin v1.0`	Mark a release baseline.

6. Application to Campus Connect

6.1 Repository structure

```
campus-connect/
|-- Campus_Connect.html      single-file prototype (HTML + CSS + JS)
|-- README.md                overview, how to run, team, licence
|-- docs/
|   |-- SRS.pdf              requirement specification (Exp. 5)
|   |-- uml/                 use case, class, sequence diagrams (Exp. 6-8)
|   |-- wireframes/          UI designs (Exp. 9)
|   |-- testing/             test plan and test cases (Exp. 10-12)
|   `-- presentation.pptx    final technical presentation
`-- .gitignore
```

6.2 Branching strategy



Feature branches merge into develop; develop merges into main for releases.

Fig. 13.1 Branches and merges of the project

Branch	Purpose	Rule
main	Stable, demonstrable releases	Only merged from develop after testing; tagged (v0.1, v1.0)
develop	Integration of finished features	Merge feature branches by pull request with one reviewer
feature/marketplace	Marketplace increment	One branch per feature; short-lived; delete after merge
feature/career	Career Connect increment	Same as above
fix/<name>	Bug fixes found in testing	Small branch from develop; reference the defect ID

6.3 Commit message convention

Format: `type: short summary`, with types *feat*, *fix*, *docs*, *test*, *style* and *refactor*. Examples: `feat: add marketplace price filter`; `fix: block duplicate applications`; `docs: add SRS v1.0`.

6.4 Sample history (illustrative)

```
$ git log --oneline --graph --all
*   a41c9e2 (HEAD -> develop) Merge branch 'feature/career' into develop
|\
| * 7d0b3a1 (feature/career) feat: recruiter updates application status
| * 52e8f90 feat: student applies with note
| * 1c77d4b feat: opportunity search by skills and type
|/
*   9be2c05 Merge branch 'feature/marketplace' into develop
|\
| * 3f1a8d6 feat: save favourites and send seller enquiry
| * c20e7b4 feat: listing create, edit, delete
|/
* 6a5d3e9 feat: navigation, theme toggle, register and login
* e01f2ab Initial commit
```

Hashes and messages above are examples to show the format. Replace this block with the output of your own repository.

6.5 Merge conflict and its resolution

Suppose Aditya (on `feature/marketplace`) and Ashish (on `feature/career`) both edit the navigation text in the same line of Campus_Connect.html. Merging the second branch stops with a conflict:

```
$ git merge feature/career
Auto-merging Campus_Connect.html
CONFLICT (content): Merge conflict in Campus_Connect.html
Automatic merge failed; fix conflicts and then commit the result.

<<<<<<< HEAD
<a href="#market">Marketplace</a>
=======
<a href="#careers">Career Connect</a>
>>>>>>> feature/career
```

Resolution steps:

1. Run `git status` to list conflicted files.
2. Open the file, decide the correct content and delete the markers (`<<<<<<<`, `=======`, `>>>>>>>`). Here both links are kept.
3. Run `git add Campus_Connect.html` to mark it resolved.
4. Run `git commit` to complete the merge, then test the page.
5. Push the result with `git push origin develop`.

6.6 GitHub workflow and .gitignore

Each feature is developed on its own branch, pushed to GitHub and merged to develop with a **pull request** reviewed by another member (see Experiment 14). Issues track defects and tasks.

```
# .gitignore
.DS_Store
Thumbs.db
node_modules/
.vscode/
*.log
```

Item	Value
Repository name	campus-connect
Repository URL	https://github.com/<team-username>/campus-connect (replace with the real link)
Default branch	main
Release tag	v1.0 (final demonstration)
Collaborators	Subham Kumar Bhoi, Aditya Shekhar, Ashish Kumar, Sovit Kumar Mahakur

7. Observations

- Feature branches keep unfinished work out of main and reduce the merge-conflict risk (R10).
- Because the prototype is a single file, conflicts are likely; small commits and frequent pulls limit them.
- Tags create baselines that can be restored if a later change breaks the demonstration.

8. Result

Git was configured and the full workflow – repository creation, commit, push, pull, branching, merge and conflict resolution – was documented for the Campus Connect repository with a defined branching strategy.

9. Viva Questions and Answers

Q1. What is version control and why is it needed?

Ans. It records changes to files over time so that versions can be compared, restored and shared, enabling safe teamwork.

Q2. Difference between Git and GitHub?

Ans. Git is the version control tool that runs locally; GitHub is a web service that hosts Git repositories and adds collaboration features.

Q3. Explain the three areas of Git.

Ans. Working directory (files you edit), staging area (changes selected for the next commit) and repository (committed history).

Q4. What is the difference between git fetch and git pull?

Ans. fetch downloads remote changes without merging; pull = fetch + merge into the current branch.

Q5. How do you resolve a merge conflict?

Ans. Edit the conflicted file to keep the right content, remove markers, git add the file and commit.

Q6. What is a baseline?

Ans. A reviewed and approved version of a work product that can be changed only through formal change control, e.g. tag v1.0.

Q7. Why use branches?

Ans. They isolate work in progress, allow parallel development and keep main stable.

EXPERIMENT 14

Software Quality Assurance

Lab week	Week 15
Course outcome	CO5
Tools used	GitHub (pull-request review), Visual Paradigm, Excel
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To apply software quality assurance practices to Campus Connect: peer review, formal technical review, defect logging, checklist preparation and ISO documentation.

2. Objectives

- Understand SQA activities and quality models.
- Conduct a peer review and a Formal Technical Review (FTR).
- Log and classify defects and compute defect metrics.
- Prepare review checklists and list applicable ISO / IEEE standards.

3. Theory

Software Quality Assurance is a set of activities that ensures processes and products meet defined standards. Activities include standards enforcement, reviews and audits, testing, defect collection and analysis, change management and reporting. **Peer reviews** are informal examinations of work by colleagues. A **Formal Technical Review (FTR)** is a structured meeting with defined roles – producer, review leader, reviewers and recorder – limited to about two hours and 3–5 people, ending in a decision: accept, accept with changes or reject. Quality models such as McCall's factors and **ISO/IEC 25010** describe characteristics like functional suitability, performance, usability, reliability, security, maintainability and portability. **Defect density** = number of defects / KLOC.

4. Software / Tools Required

- GitHub pull requests and issues for peer review; Excel for defect log and checklists; Word for review reports.

5. Procedure

1. Prepare checklists for documents and code.
2. Conduct a peer review through a pull request.
3. Hold an FTR meeting and record decisions.
4. Log defects with severity and corrective action; compute metrics.
5. Evaluate quality characteristics and list applicable standards.

6. Application to Campus Connect

6.1 Peer review checklist (code and documents)

#	Check	Applies to
1	Code follows naming and indentation conventions; functions are short and commented.	Code
2	All user input is validated and escaped before display.	Code
3	Each requirement (FR / NFR) is implemented and traceable to a test case.	Code / SRS
4	No duplicate or dead code; storage is accessed only through the helper.	Code
5	Error messages are clear; empty states and confirmations exist.	UI
6	Page works at 360 px and in both themes; labels and contrast are adequate.	UI
7	Diagrams are consistent with the SRS and with each other.	Design
8	Documents use the standard template; version and date are present.	Documents

6.2 Formal Technical Review record

Item reviewed	Campus_Connect.html (marketplace and career modules) and SRS v1.0
Review leader / recorder	Subham Kumar Bhoi (leader); Sovit Kumar Mahakur (recorder)
Producers / reviewers	Aditya Shekhar, Ashish Kumar (producers); all members and the guide (reviewers)
Duration	2 hours; material distributed two days in advance
Decision	Accept with modifications – defects below to be fixed and re-checked by the review leader

Names, roles and dates in this record are suggestions. Replace them with the real review details of your team.

6.3 Defect log (illustrative)

ID	Defect	Type	Severity	Corrective action
D-01	Credentials kept as plain text in browser storage	Security	High	Documented limitation; hash on server in next phase
D-02	User text inserted into the page without escaping	Security	High	Escape output or use textContent
D-03	Price field accepts negative or non-numeric values	Functional	Medium	Add numeric range validation
D-04	Same student can apply twice to one opportunity	Functional	Medium	Check existing application before saving
D-05	Delete listing has no confirmation	Usability	Low	Add confirm dialog
D-06	Status chip contrast is low in dark theme	Accessibility	Low	Adjust colours to meet contrast ratio
D-07	Skill search is case-sensitive	Functional	Low	Compare in lower case
D-08	No message when filters return no results	Usability	Low	Add empty-state text
D-09	All code in one file is hard to review	Maintainability	Medium	Group by module; consider splitting files
D-10	Dates shown in raw format on dashboard	UI	Low	Format dates for readability

This defect log is illustrative: it reflects typical findings for a browser-only prototype. Replace or confirm each entry with the defects your team actually found during review.

6.4 Defect metrics

Metric	Calculation	Value
Defects found		10
By severity	High / Medium / Low	2 / 3 / 5
Defect density	10 defects / 4.77 KLOC	2.1 defects per KLOC
Review effectiveness	Defects found in review / total found (review + test)	Record after testing

6.5 Quality characteristics (ISO/IEC 25010) checklist

Characteristic	Evidence in Campus Connect	Assessment
Functional suitability	All 21 functional requirements traced to test cases	Satisfactory
Performance efficiency	In-browser filtering of sample data is immediate	Satisfactory for prototype
Usability	Shared navigation, feedback messages, light / dark theme	Good
Reliability	Data persists in browser; loss possible if storage cleared	Needs backend
Security	Plain-text credentials; limited validation	Weak – improve
Maintainability	Single file; modular comments	Fair
Portability	Runs in modern browsers, responsive layout	Good

6.6 ISO / IEEE documentation applicable to the project

Standard	Use in Campus Connect
ISO/IEC/IEEE 12207	Software life-cycle processes – basis of the incremental process (Exp. 1)
IEEE 1058	Software project management plan (Exp. 2)
IEEE 830 / ISO/IEC/IEEE 29148	Software requirements specification (Exp. 5)
ISO/IEC 19505 (UML)	Modelling notation (Exp. 6–8)
ISO/IEC/IEEE 29119 and IEEE 829	Software testing and test documentation (Exp. 10–12)
ISO/IEC 25010	Product quality model used for evaluation
ISO 9001	General quality management system principles (context for SQA)

6.7 SQA plan outline

Activity	Responsibility	Frequency
Peer review of each pull request	Another team member	Every merge
FTR of SRS and design	Review leader	Once per increment
Testing and defect tracking in GitHub issues	Test lead	Continuous
Process audit against this plan	Guide	Mid-term and final

7. Observations

- Security-related defects dominate the high-severity list, in line with risk R4 identified in Experiment 4.
- Defect density of the illustrative log is 2.1 per KLOC, which will change once real review data is recorded.
- Using pull requests makes peer review part of the normal workflow rather than an extra step.

8. Result

Peer review, FTR, defect logging, checklists and ISO documentation mapping were prepared for Campus Connect and quality metrics were computed.

9. Viva Questions and Answers

Q1. Difference between quality assurance and quality control?

Ans. QA is process-oriented and prevents defects; QC is product-oriented and detects defects through inspection and testing.

Q2. What is an FTR and who takes part?

Ans. A structured review of a work product led by a review leader with producer, reviewers and a recorder; it ends with accept, modify or reject.

Q3. Name some guidelines for conducting an FTR.

Ans. Review the product, not the producer; set an agenda and keep to it; limit debate; take notes; 3–5 participants; under two hours; prepare in advance.

Q4. How is defect density calculated?

Ans. Number of defects divided by size in KLOC.

Q5. What are ISO/IEC 25010 quality characteristics?

Ans. Functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability and portability.

Q6. Why is peer review cheaper than testing for finding defects?

Ans. Defects are found at the source early, before code is integrated, so fixing cost is lower.

Q7. What is the role of ISO 9001 in software?

Ans. It sets general requirements for a quality management system; ISO/IEC 90003 gives guidance for applying it to software.

EXPERIMENT 15

Mini Project – Documentation and Presentation

Lab week	Weeks 1 – 15 (final demonstration in Week 15)
Course outcome	CO1 – CO5
Tools used	HTML, CSS, JavaScript, Git / GitHub, UML tools, Figma
Project	Campus Connect – A Unified Platform for Campus Marketplace and Career Development (Team FADED)

1. Aim

To document the Campus Connect mini project that applies the complete software engineering life cycle, and to prepare its final technical presentation.

2. Objectives

- Identify a real-world problem and define the solution.
- Link every life-cycle activity to the artefact produced in the lab experiments.
- Document implementation, testing, limitations and future scope.
- Prepare the research article abstract and the final presentation.

3. Theory

A software project passes through problem identification, planning, requirements analysis, modelling, design, coding, testing, configuration control and quality assurance. Good documentation allows others to understand, maintain and extend the system. A **technical presentation** explains the problem, solution, process followed, results and lessons within a fixed time.

4. Software / Tools Required

- VS Code, a web browser, Git and GitHub, StarUML / Visual Paradigm, Figma, PowerPoint.

5. Procedure

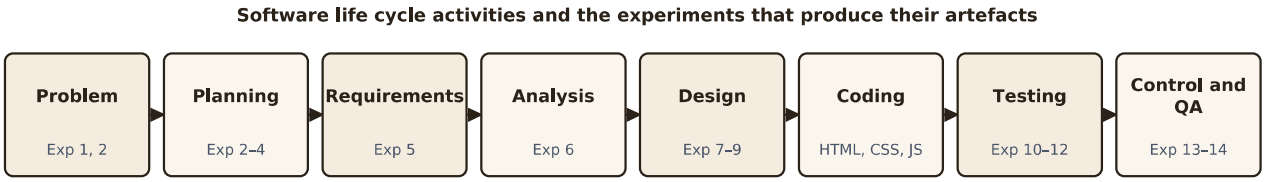
1. Define the problem from real student needs.
2. Plan, specify, model and design (Experiments 1–9).
3. Implement the prototype in increments.
4. Test (Experiments 10–12), control versions (Experiment 13) and review quality (Experiment 14).
5. Prepare the report, article abstract and presentation; demonstrate and answer viva questions.

6. Application to Campus Connect

6.1 Project summary

Title	Campus Connect – A Unified Platform for Campus Marketplace and Career Development
Institution	Central University of Karnataka, Kalaburagi, Karnataka
Guide	Dr. Rohit Kumar B., Assistant Professor, School of Computer Science
Team	TEAM FADED – Subham Kumar Bhoi (Lead), Aditya Shekhar, Ashish Kumar, Sovit Kumar Mahakur
Problem	Student needs are split across disconnected platforms: resale is scattered, career discovery is fragmented, and users need clear profiles, manageable listings, application status and feedback.
Solution	One responsive web interface with a marketplace, career listings, shared accounts, dashboards and theme preferences.
Technology	HTML + CSS for responsive pages, cards and forms; JavaScript for routing, filters, forms and state; browser storage for persistence.

6.2 Life-cycle traceability



Mini project follows the complete life cycle; the final presentation closes the loop.

Fig. 15.1 Life-cycle activities mapped to the lab experiments

Activity	Artefact in this record	Experiment
Process selection	Comparison report, incremental life cycle	1
Planning	SPMP, WBS, Gantt chart, milestones	2
Estimation	LOC, FP and COCOMO: 12.4 PM, 6.5 months	3
Risk analysis	Risk register, risk matrix, RMMM plan	4
Requirements	IEEE SRS with 21 FR, 7 NFR, 4 use cases	5
Analysis and design	UML behavioural and structural models, architecture, wireframes	6–9
Coding	Campus_Connect.html (single-file prototype)	Implementation
Testing	Test plan, 24 test cases, black-box and white-box tests	10–12
Configuration management	Git repository, branches, tags	13
Quality assurance	Peer review, FTR, defect log, ISO mapping	14

6.3 Implemented features

Module	Features
Campus Marketplace	Browse by keyword, category and condition; filter by maximum price; sort newest or by price; create, edit and delete personal listings; save favourites; contact sellers. Sample categories: Books, Electronics, Furniture, Cycles, Clothing, Stationery.
Career Connect	Opportunity types: Internship, Full-time, Capstone project, Mentorship. Students search by role, company, skills and type, apply with a note and track status (Submitted, Shortlisted, Interview, Rejected). Recruiters post, edit and remove roles, review applicants and update status.
Shared features	Account registration, profile management (education / company affiliation, skills), notifications for enquiries and application updates, dashboard (my listings, saved items, applications, summary counts), light / dark theme, responsive layout, accessible labels and feedback.

6.4 Current boundaries and limitations

- Accounts and records are stored locally in the browser and are not shared across users.
- No server-side database, real university verification, file uploads, payments or real-time chat.
- Sample listings and opportunities are demonstration data.

6.5 Future scope

Next step	Description
Backend and database	Persist accounts, listings, jobs and applications centrally.
Real identity and access	Secure server-side authentication and campus / company verification.
Richer interactions	Image uploads, in-app messaging, moderation and reporting.
Deploy and evaluate	Test accessibility, security, usability and multi-device behaviour with real users.

6.6 Research article outline (Mini Project assessment)

Title	Campus Connect: A Unified Web Platform for Campus Resale and Early-Career Opportunity Management
Abstract	Students often depend on scattered channels to buy second-hand goods and to find internships, mentorships and projects. This paper presents Campus Connect, a responsive web prototype that integrates a campus marketplace and a career portal with shared accounts, dashboards and notifications. The system was developed with an incremental, agile-style life-cycle model, specified through an IEEE-format SRS, modelled in UML, estimated at about 12.4 person-months using COCOMO and 158 function points, and verified with black-box and white-box test techniques under Git-based configuration management. The paper discusses the design, the quality assurance practices applied and the limitations of a browser-only prototype, and outlines a path to a campus-ready platform.
Keywords	campus marketplace; career portal; software life cycle; incremental model; UML; COCOMO; testing
Sections	1 Introduction; 2 Related work; 3 Process model (LCM) and planning; 4 Requirements and design; 5 Implementation; 6 Testing and quality assurance; 7 Results and discussion; 8 Limitations and future work; 9 Conclusion; References

6.7 Final presentation plan (about 12 minutes)

Slide	Content	Time
1 Title	Project, team, guide, university	0.5 min
2 What is Campus Connect	Disconnected platforms; our response	1.5 min
3 Product overview	Two modules, one consistent experience; shared features	1.5 min
4 Marketplace	Discover, compare, list and manage, connect	2 min
5 Career portal	Opportunity types; student and recruiter flows; status tracking	2 min
6 Implementation and scope	HTML, CSS, JavaScript, browser storage; prototype boundaries	1.5 min
7 Next steps	Backend, identity, messaging, deployment	1 min
Demo and viva	Live walk-through: list an item, apply, shortlist, notification	2 min

6.8 Demonstration script

1. Open Campus_Connect.html; switch between light and dark themes.
2. Register a student and a recruiter.
3. As student: search items, filter by price, create a listing, save an item and send an enquiry.
4. As recruiter: post an opportunity.
5. As student: search the opportunity, apply with a note.
6. As recruiter: review the applicant and set status to Shortlisted.
7. As student: open notifications and dashboard to see the update.

7. Observations

- Every phase of the life cycle produced a traceable artefact, so requirements link to design and tests.
- An incremental approach made the prototype demonstrable early and kept the scope under control.
- The main learning was the value of planning, modelling and testing before and during coding, and the cost of weak security and storage choices in a browser-only design.

8. Result

The Campus Connect mini project was documented across the complete software engineering life cycle, and its research article outline and final presentation plan were prepared.

Conclusion

Campus Connect shows how a small team can use software engineering practices – process selection, planning, estimation, risk analysis, specification, modelling, testing, configuration management and quality assurance – to deliver a coherent product. The next phase should add a backend, real identity and richer interactions to turn the prototype into a campus-ready platform.

References

1. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
2. I. Sommerville, *Software Engineering*, Pearson.
3. B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981.
4. A. J. Albrecht and J. E. Gaffney, "Software function, source lines of code and development effort prediction," *IEEE Trans. Software Engineering*, 1983.
5. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*; ISO/IEC/IEEE 29148:2018.
6. ISO/IEC 25010:2011, *Systems and software quality models*.
7. Campus Connect project presentation, Team FADED, Central University of Karnataka.

9. Viva Questions and Answers

Q1. What problem does Campus Connect solve?

Ans. It removes the fragmentation of campus resale and career discovery by offering a marketplace and a career portal in one platform with profiles, dashboards and status tracking.

Q2. Which process model did you use and why?

Ans. Incremental with Agile practices, because requirements evolved, early demos were valuable and the team was small with a fixed semester.

Q3. How did you estimate effort?

Ans. Bottom-up LOC (4.77 KLOC), function points (157.6 FP) and Basic COCOMO organic mode giving 12.4 person-months.

Q4. How did you test the system?

Ans. Using a test plan with 24 test cases, black-box techniques (BVA, equivalence partitioning, decision table, state transition) and white-box basis path testing of the filter logic.

Q5. What are the limitations of the prototype?

Ans. Data is stored in the browser only, with no server database, verification, uploads, payments or real-time chat.

Q6. How did you manage versions and quality?

Ans. Git and GitHub with feature branches, pull-request review, tags, FTR and a defect log.

Q7. What would you do next?

Ans. Build a backend with database, secure authentication and verification, add image uploads and messaging, then evaluate with real users.

Team Member Status

Track each member's task, progress, and deadline

Member Name	Role	Assigned Task	Status	Progress (%)	Deadline	Remarks
Ashish kumar	Dashboard,ppt	build task tracker, ppt for project ,	Completed	100%	2026-11-15	completed
Aditya shekhar	Prototype	Choose prototype , backend	In Progress	100%	2026-11-15	completed
Subham Bhoi	SRS	Build srs	Completed	100%	2026-11-15	completed
Sovit	Reaserch for ppt, srs	for project, backend	Completed	100%	2026-11-15	completed
Ashish kumar	working on backend	server, database , frontend	Completed	100%	2026-11-15	completed
Aditya shekhar	frontent	design , backend	Completed	100%	2026-11-15	comleted
Sovit	log in credential	log in Aunthentication , backend	Completed	100%	2026-11-15	completed
Subham Bhoi	prepared data	data analysis	Completed	100%	2026-11-15	completed

Note: Yellow cells are editable. Use the dropdown in Status column. Progress is entered as a percentage (e.g., 60%).

FIGURE 1.1 : MVP LANDING PAGE

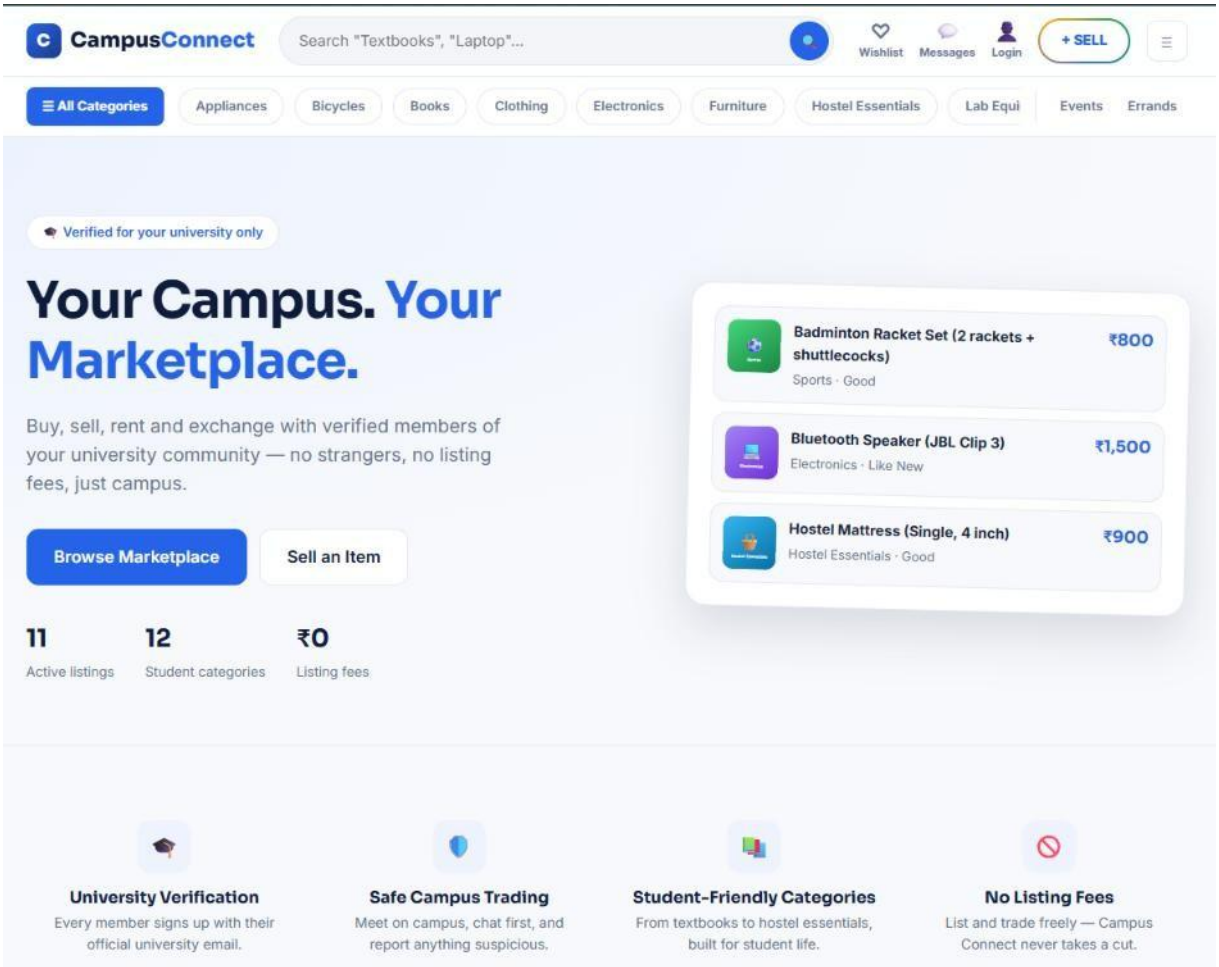


FIGURE 1.2 : LOGIN CREDENTIALS

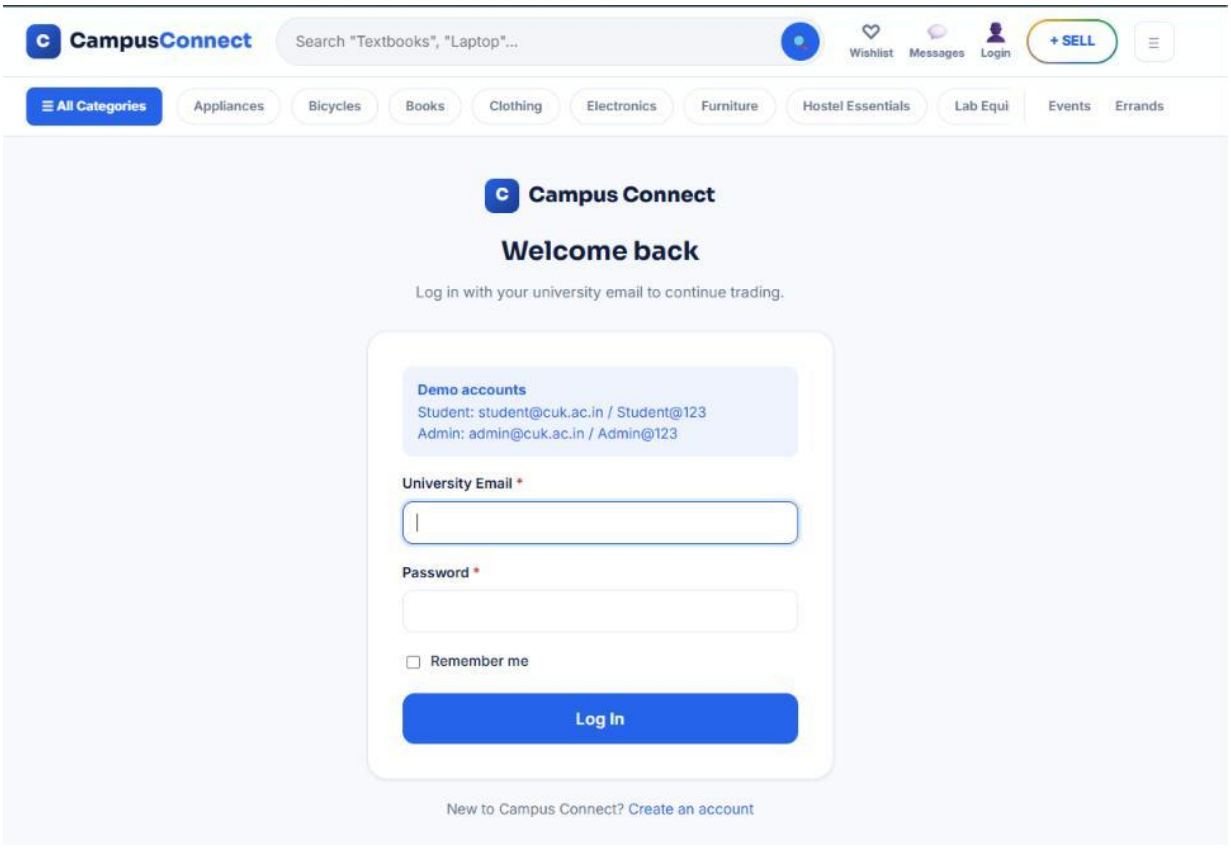


FIGURE 1.3: MARKETPLACE

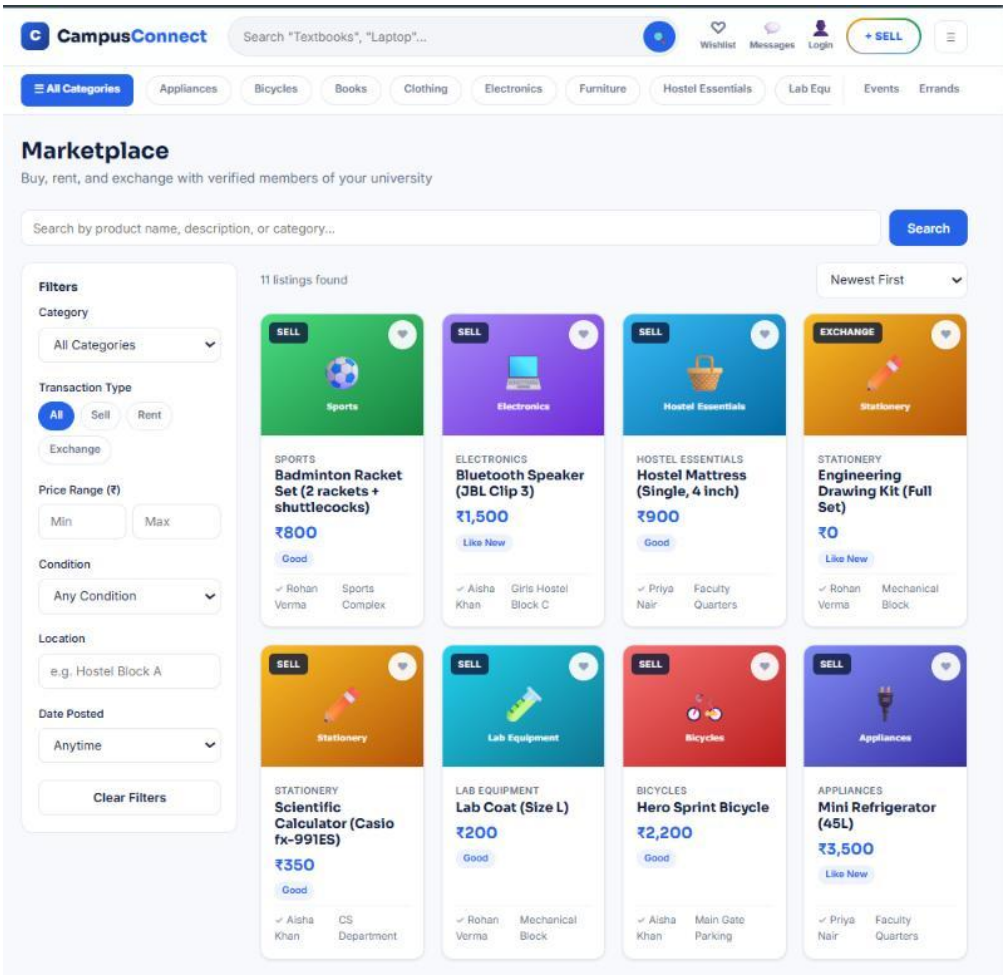


FIGURE 1.4: ABOUT US

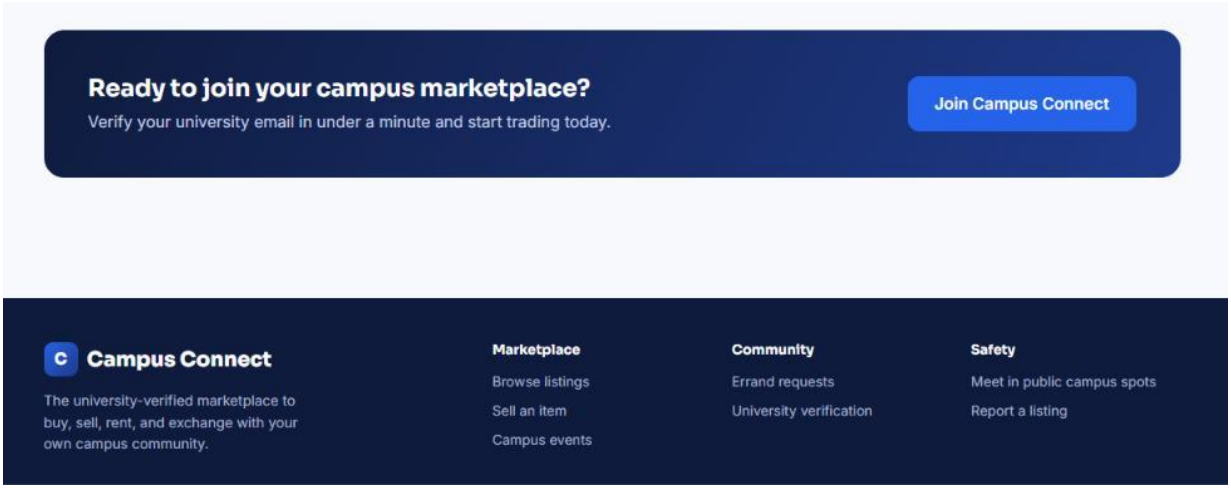


FIGURE 1.5: PRODUCTS LISTING FEATURE:

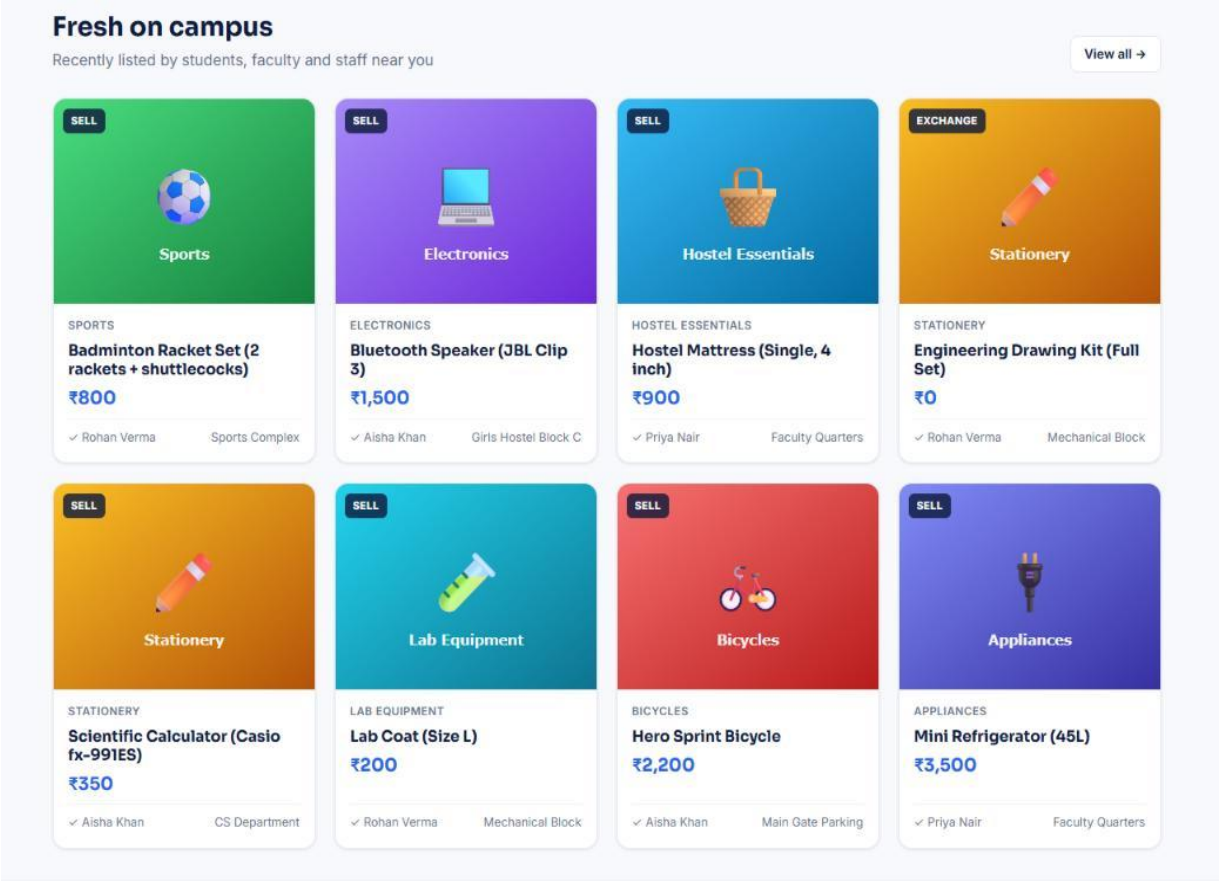


FIGURE 1.6: PRODUCT LISTING : PYTHON BOOK

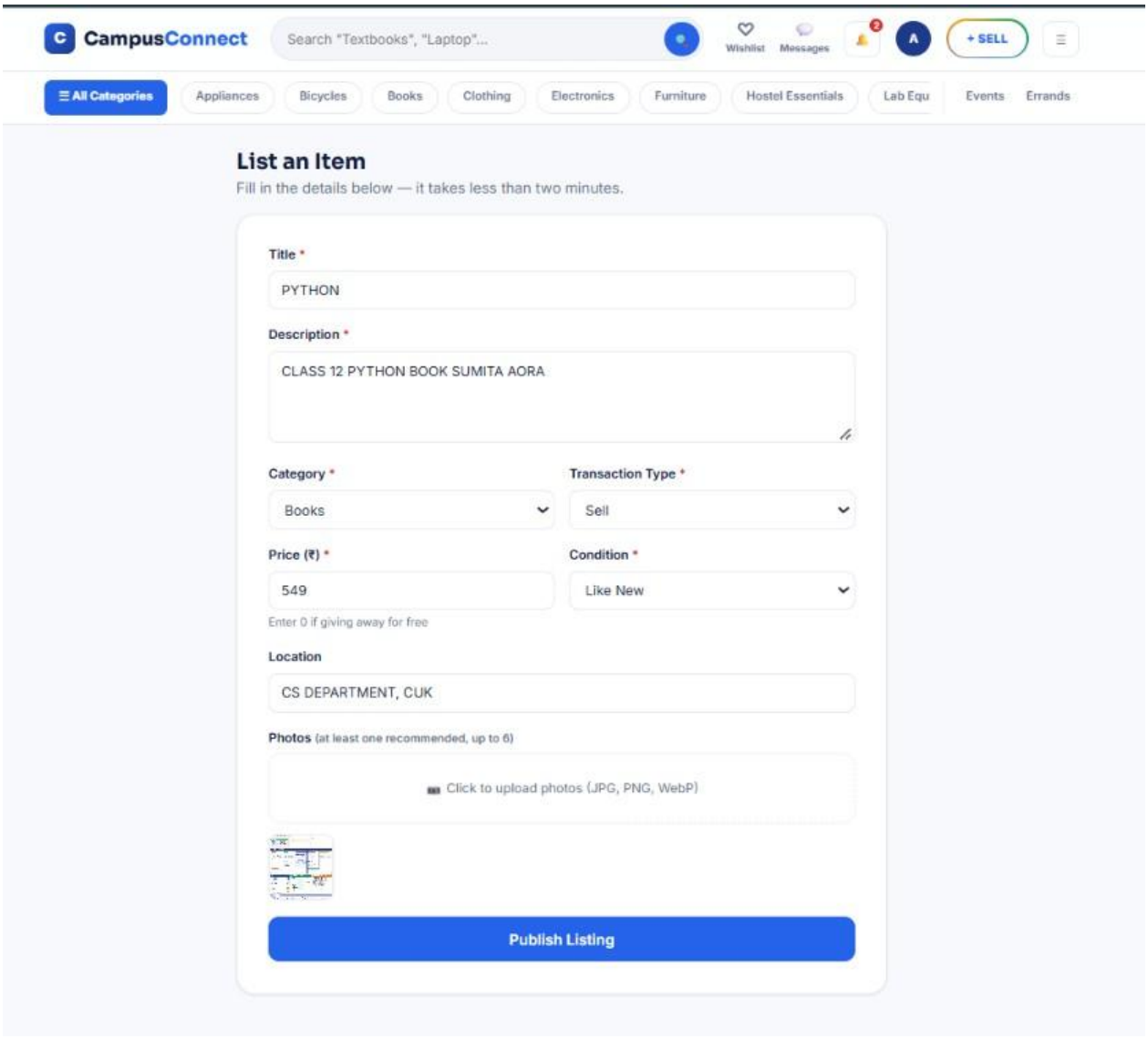


FIGURE 1.7: BUYER & SELLER CHAT FEATURE

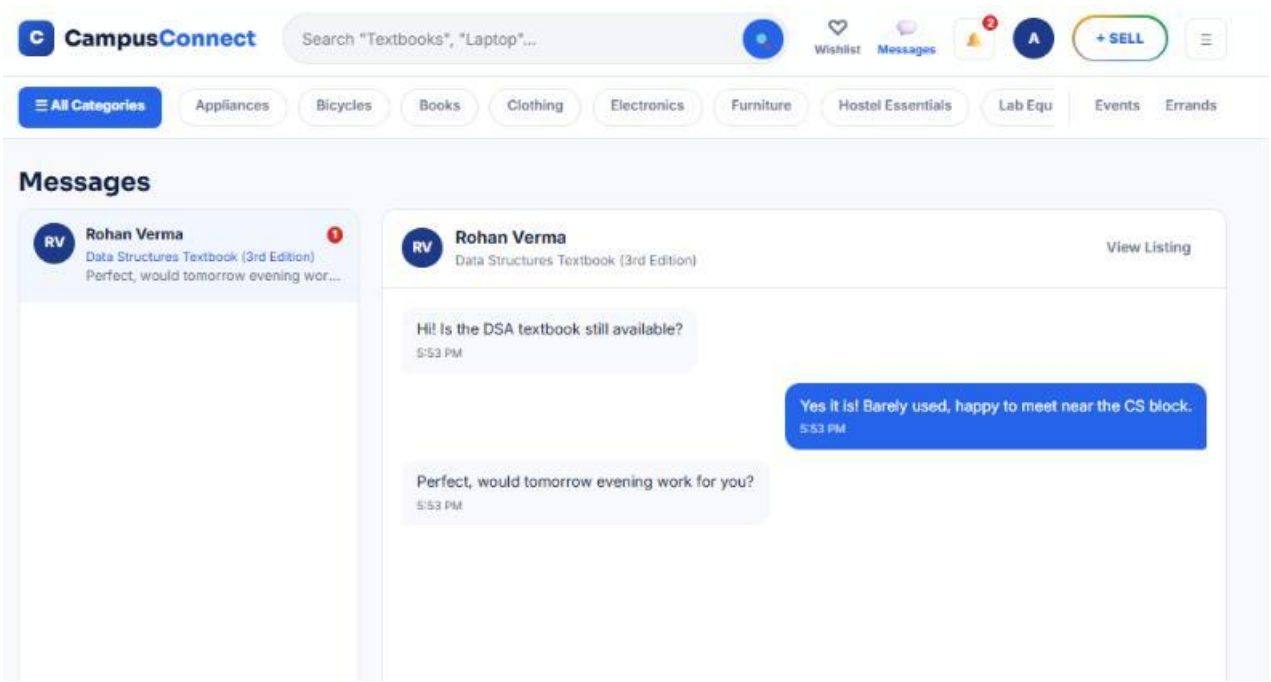


FIGURE 1.8 : UNIVERSITY EVENT LISTING FEATURE:

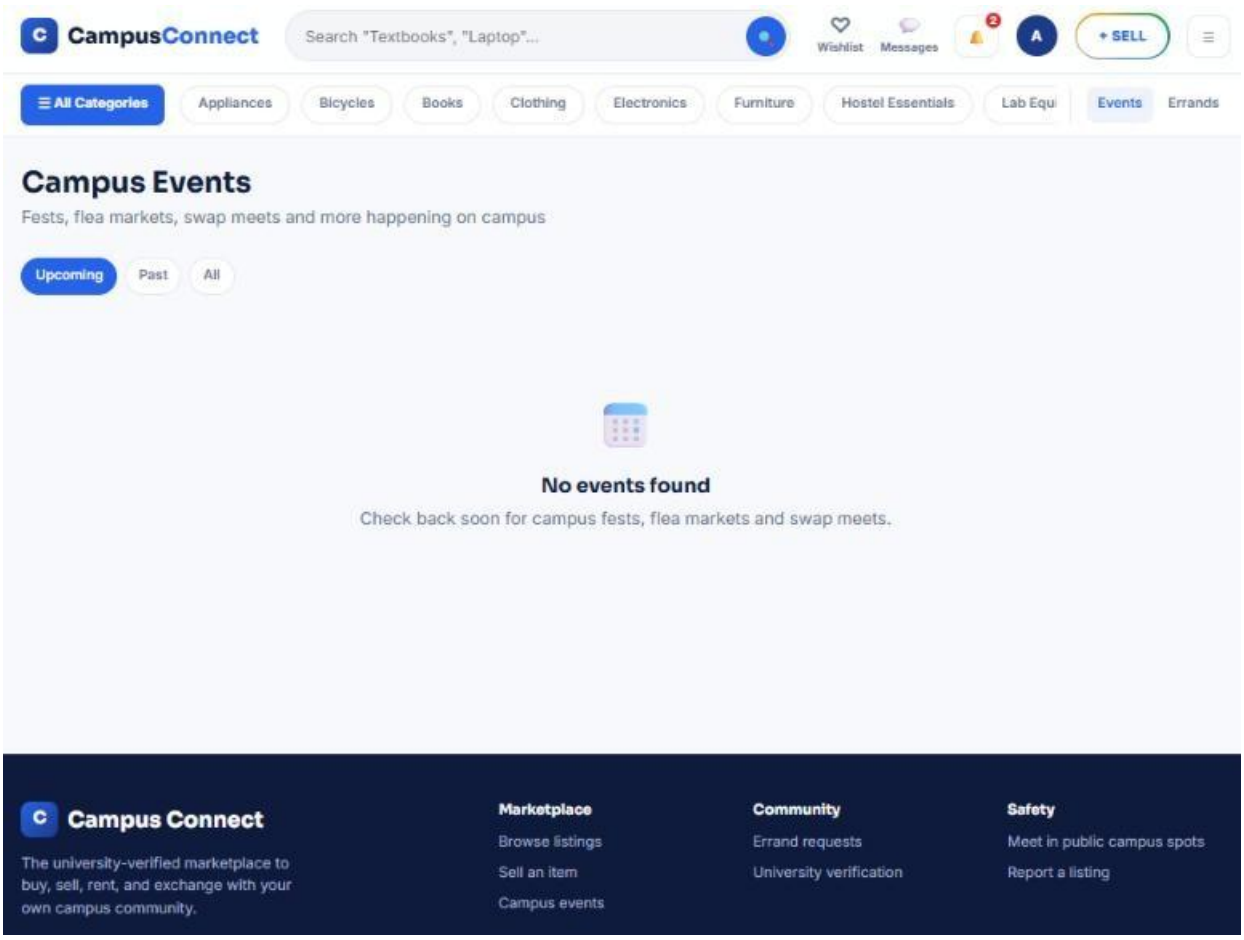
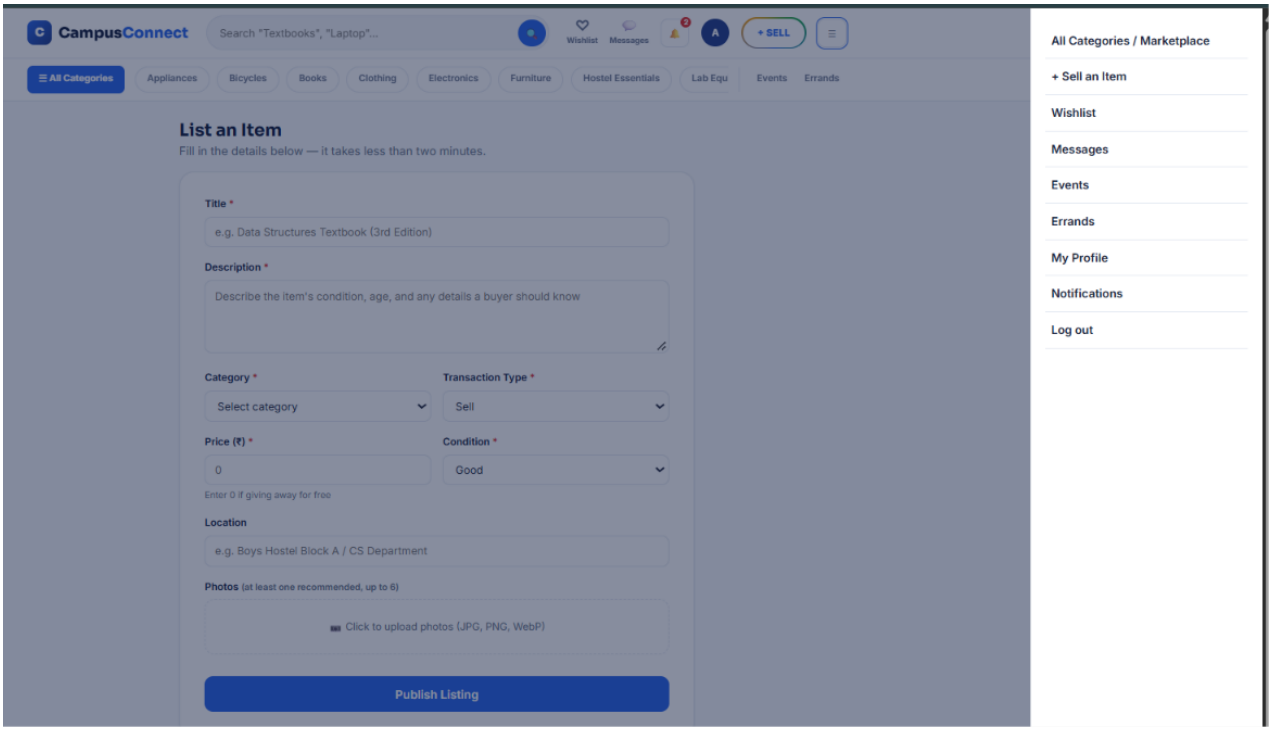


FIGURE 1.9: QUICK ACCESS FEATURE



LEAN CANVAS MODEL : CAMPUS CONNECT

कर्नाटक केंद्रीय विश्वविद्यालय

CENTRAL UNIVERSITY OF KARNATAKA

Kalaburagi, Karnataka

CAMPUS CONNECT

CAMPUS SECOND-HAND MARKET • BUY • SELL • RENT • EXCHANGE

INNOVATEX HUB

UNDER THE GUIDANCE OF

DR. ROHIT KUMAR

TEAM NAME: TEAM FADED

Subham Kumar Bhoi (Team Lead)
Aditya Shekhar
Ashish Kumar
Sovit Kumar Mahakur

1 INTRODUCTION

A university-verified portal that works as a second-hand market where campus members buy, sell, rent and exchange items such as textbooks, lab equipment and hostel essentials, safely within their own community.

Second-hand · Buy & Sell · Rent · Verified

! PROBLEM

- Hard to find affordable textbooks, lab and hostel items
- No organised way to resell or exchange after a term
- Notice boards and chat groups are unsearchable and unverified
- No trusted way to rent items for a fixed period

Alternatives: WhatsApp / Telegram groups, notice boards, generic marketplaces

SOLUTION

- Verified campus second-hand market
- Sell, rent and exchange with chat
- Ratings, reports, admin console
- Rental reminders and exchange offers

Key Metrics

- Verified active users
- Listings and completed deals
- Rental on-time return rate
- Exchange offers accepted

★ KEY FEATURES

1. **Verified Campus Accounts** — Only members with a university email can list, message or transact.
2. **Second-Hand Listings** — Post items to sell, rent or exchange with photos, condition and price.
3. **Search & Wishlist** — Find items by keyword, category, price and condition, and save favourites for later.
4. **Rentals & Exchange** — Agree a rental period with due-date reminders, or swap items through offers.
5. **In-App Chat** — Talk to sellers or renters securely without sharing contact details first.
6. **Ratings & Reports** — Rate each other after a deal and report suspicious listings or users.
7. **Admin Console** — Admins verify users, review reports and keep audit logs.
8. **Reminders & Notifications** — Get alerts for new messages, rental due dates and offers.

4 INNOVATION & UNIQUENESS

Feature	Campus Connect	Chat Groups	Notice Board	Generic Market
Verified campus-only users	✓	×	×	Limited
Second-hand buy / sell	✓	Limited	Limited	✓
Rent & exchange with reminders	✓	×	×	×
Search, filters, ratings	✓	Limited	×	Partial
Multi-institution network	Planned	×	×	Open

1 CAMPUS SECOND-HAND MARKET

(buy • sell • rent • exchange)

- **⇩ Buy** — Find pre-owned textbooks, electronics and hostel items from verified peers.
- **⇧ Sell** — List items you no longer need with photos, condition and price.
- **↻ Rent** — Borrow items for a fixed period, with due-date reminders for both sides.
- **⇄ Exchange** — Swap items directly, with accept, reject or counter offers.

1 Verify University Email → 2 List Item → 3 Search & Discover → 4 Chat with Owner → 5 Buy / Rent / Swap → 6 Rate & Review

Trust & safety built in

- University-email verification before listing, messaging or transacting
- Ratings after every deal
- Reports for suspicious listings or users, reviewed by admins
- In-app chat before any contact details are shared

Categories: textbooks, electronics, hostel essentials, lab equipment, bicycles, sports items. Only verified members can list, message or transact; payment can be in-app or off-platform.

</> TECHNOLOGY STACK

Frontend	HTML, CSS, JavaScript, React
Backend	REST API (runtime to be finalised)
Database	MongoDB (NoSQL)
Real-time chat	WebSocket
Notifications	Push (e.g. FCM), SMTP email
Storage	Cloud object storage for images
Payments	Optional gateway (e.g. UPI), TBD
Security	HTTPS/TLS, encrypted credentials, audit logs

6 VALUE PROPOSITION

- Cheaper study material through a campus second-hand market
- Less waste, more reuse of resources
- Safe deals through ratings and moderation
- Rent or swap items for a term instead of buying new

REUSE. SAVE. SHARE. →

7 SDGS ALIGNMENT

12 Responsible Consumption

4 Quality Education

Reuse of resources and affordable study materials.

8 TARGET USERS

Students · Faculty & Staff

UNIQUE VALUE PROPOSITION

The trusted second-hand market for your campus.

Buy, sell, rent and exchange with verified peers, with ratings and moderation alongside.

UNFAIR ADVANTAGE

- Closed, email-verified community
- Team lives the problem as students
- Rentals and exchange built in from day one

CHANNELS

- University email & orientation
- Hostel / department groups
- QR posters on campus
- Student clubs and campus notice boards

CUSTOMER SEGMENTS

- Students (primary)
- Faculty / staff
- Admins (moderation)

Early adopters: first-year and final-year students at term start / end

COST STRUCTURE

- Cloud hosting & image storage
- Push notifications & SMTP
- Payment gateway fees (optional)
- Development and moderation

REVENUE STREAMS (PROPOSED)

- Free for verified students
- Featured listings; small in-app payment fee
- Campus-partner sponsorships

9 CHALLENGES

- Building initial trust and adoption
- Moderating listings and disputes (policy TBD)
- Data privacy and compliance
- Choosing payment gateway and hosting
- Verifying campus accounts
- Enforcing rental returns and due dates

10 FUTURE SCOPE: ONE PORTAL, MANY INSTITUTIONS



- Connect multiple universities, NITs and IITs in a single portal
- Inter-campus second-hand market and resource sharing
- Shared listings and ratings across campuses
- Cross-campus rentals and exchange

Phase 1 — CUK pilot: verified second-hand market, chat, ratings, admin

Phase 2 — Rental and exchange refinements, optional in-app payments

Phase 3 — Multi-university, NIT and IIT network

11 OUTCOME

A trusted, university-verified platform where students save money by buying, selling, renting and exchanging items on a campus second-hand market. It starts at Central University of Karnataka and grows into a shared network linking universities, NITs and IITs.

SECOND-HAND MARKET • BUY • SELL • RENT • EXCHANGE • VERIFIED • CONNECTED CAMPUSES